Make Your Code Faster with Data Parallelism

Daniel Lemire, professor

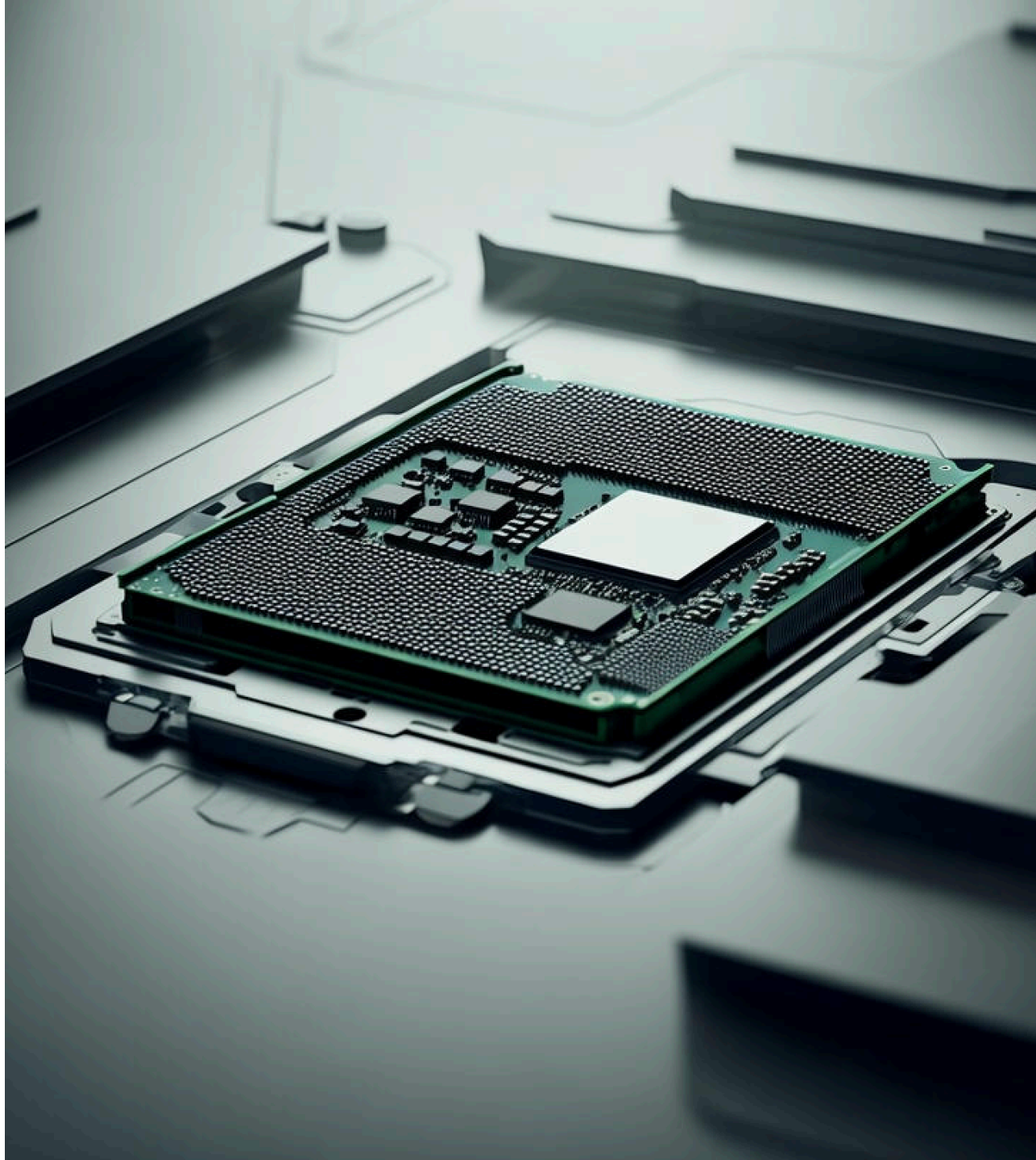
Université du Québec (TÉLUQ)

Montréal 🇨🇦

blog: https://lemire.me

X: @lemire

GitHub: https://github.com/lemire/



Where I am coming from

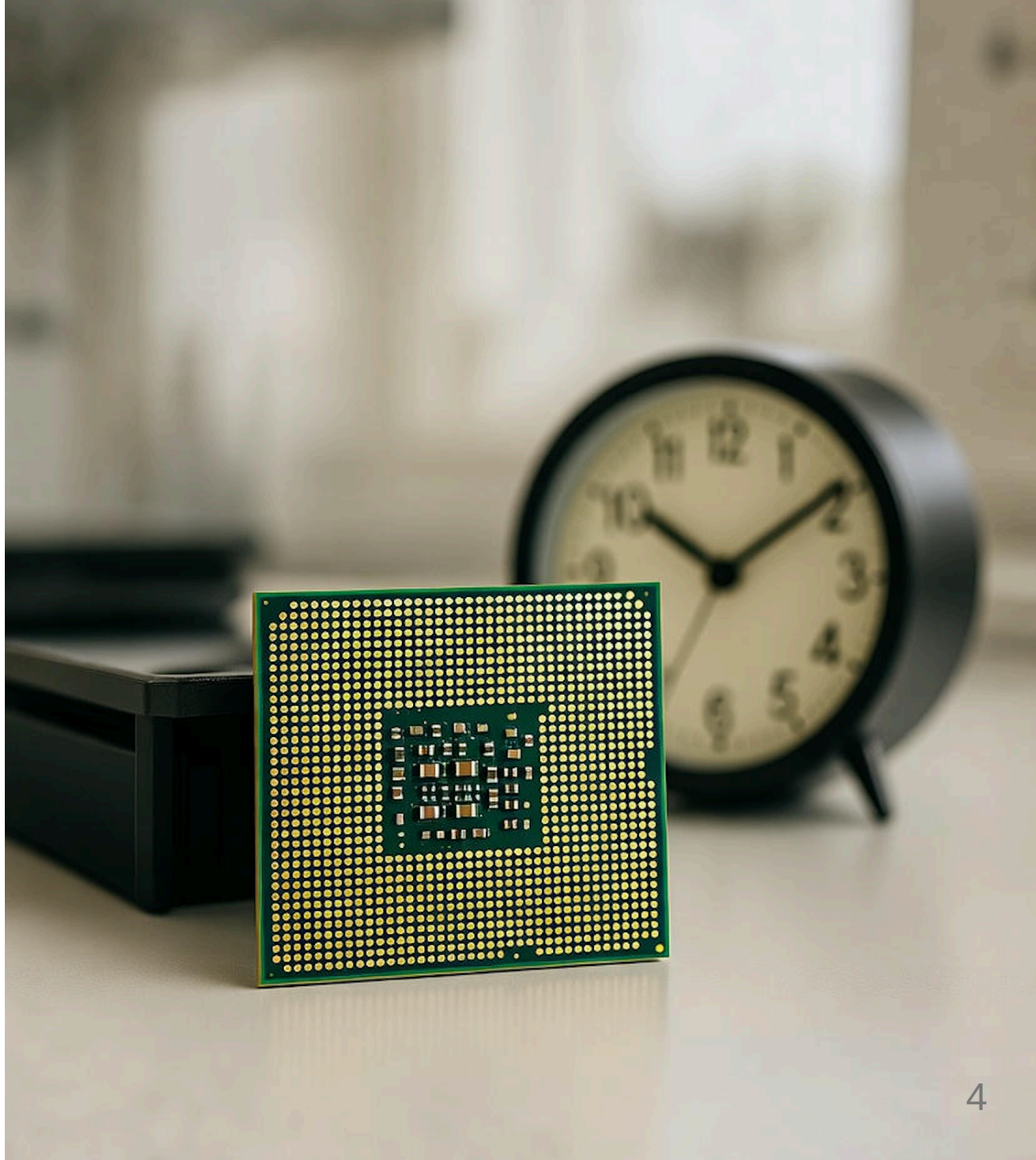
- Author of libraries you are probably running right now: **simdjson**, **simdutf**, **fast_float**, **Roaring Bitmaps**.
- Shipped inside .NET, Rust, Go, Node.js, Bun, Deno, Google Chrome, Safari, ClickHouse.
- Node.js core contributor (C++ review, performance, security).
- Editor, *Software: Practice and Experience* (Wiley, founded 1971).

Part 1

Parallelism for the win

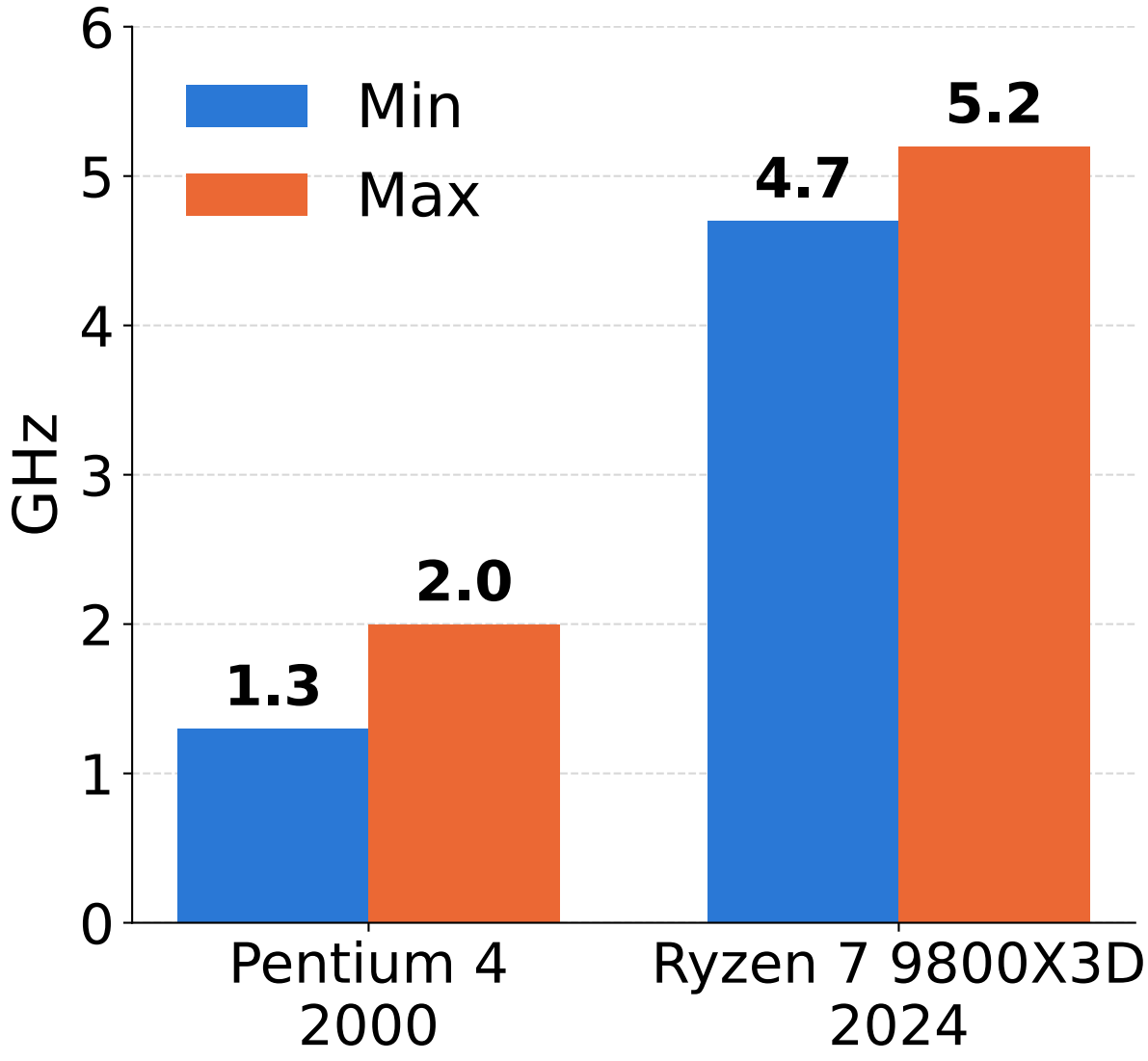
Some numbers

- Time is discrete: the clock cycle
- Processors: ~4 GHz
- One cycle is 0.25 nanoseconds
- Light travels 7.5 cm per cycle
- **One byte per cycle is only 4 GB/s**



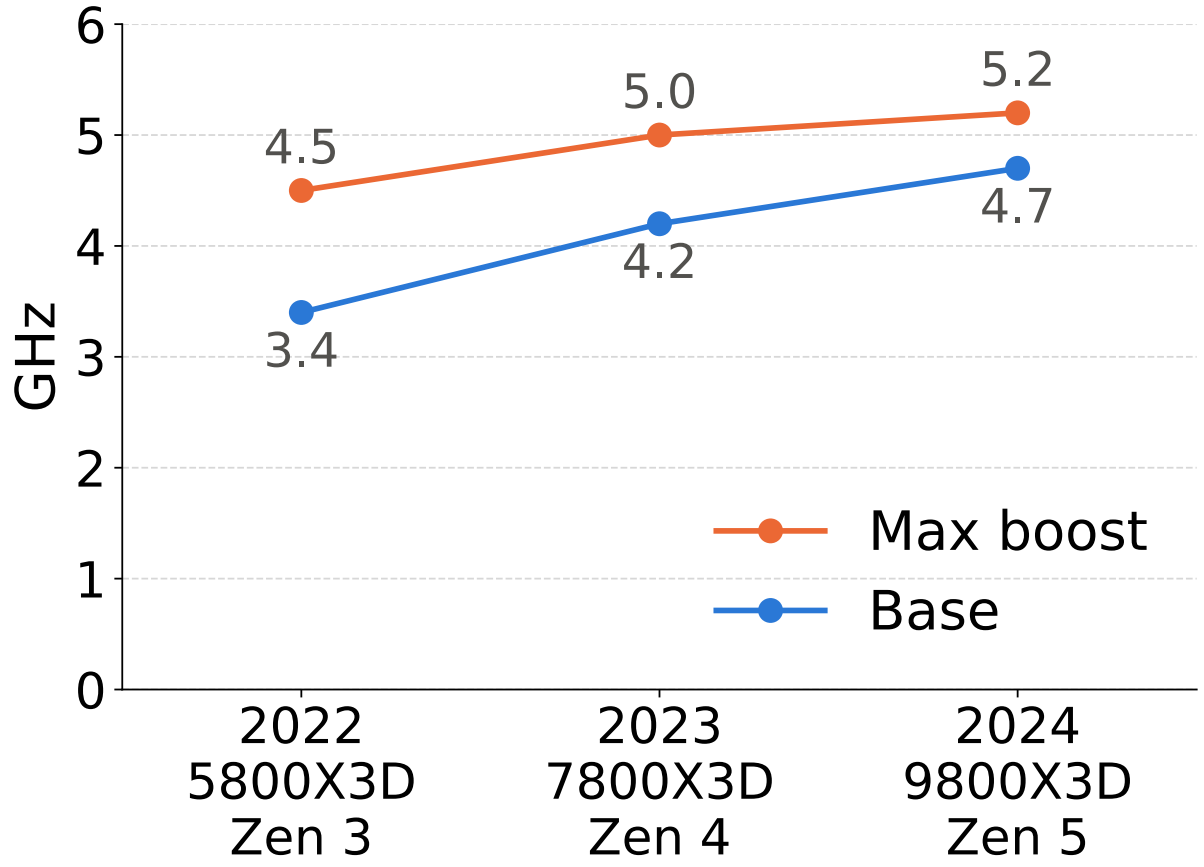
Clock frequency: 25 years

- Pentium 4 (2000): 1.3 to 2 GHz
- AMD Ryzen 7 9800X3D (2024): 4.7 to 5.2 GHz
- **About 2.5× in 25 years**

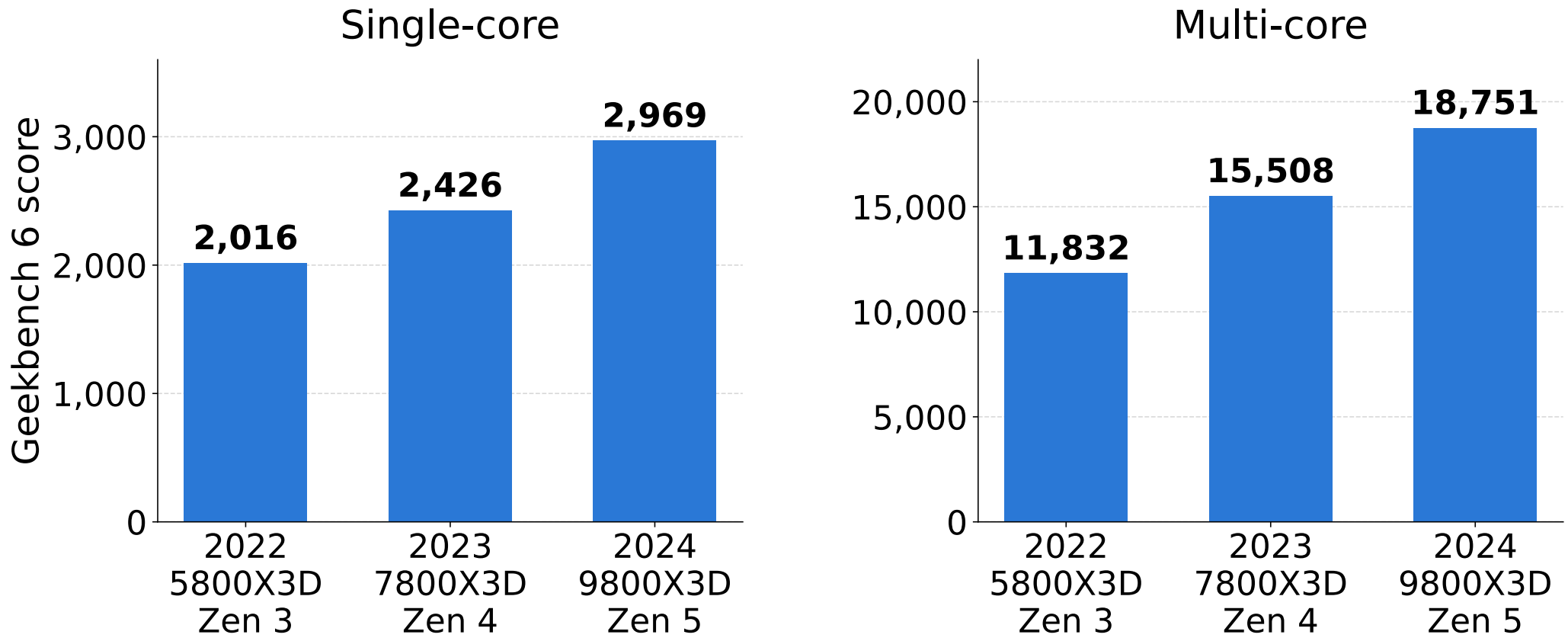


Clock frequency: AMD Ryzen 7, 2022–2024

- Up 15% is two years



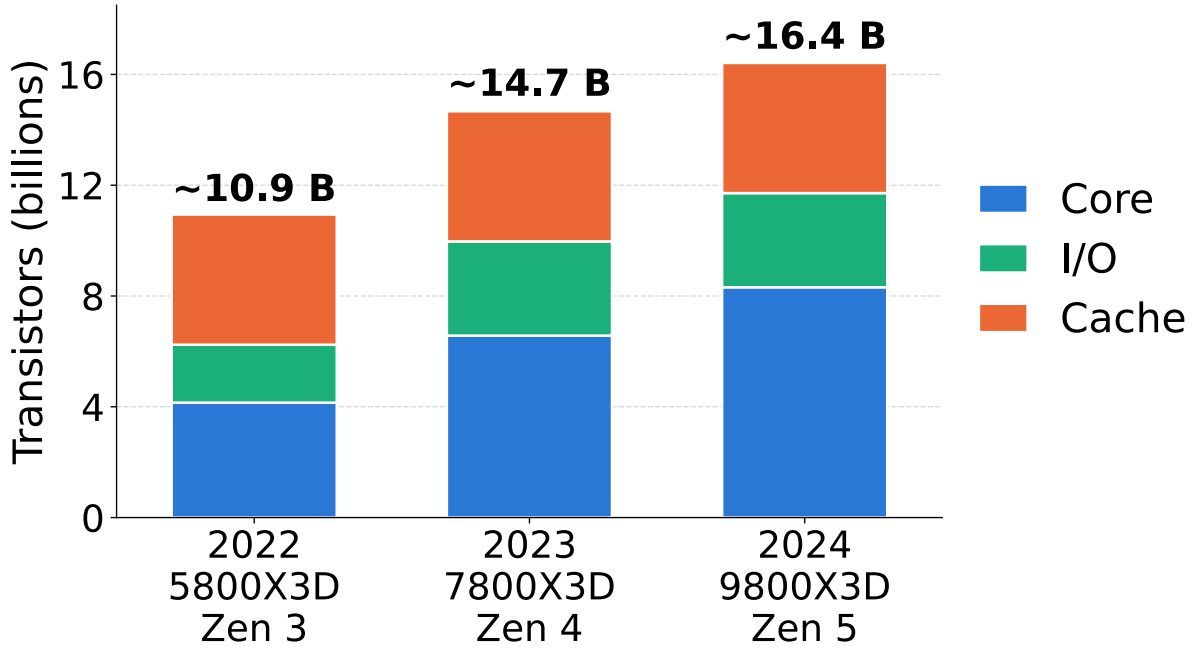
Geekbench 6: AMD Ryzen 7, 2022–2024



Single-core: **+47%**. Multi-core: **+58%**. Same 8 cores.

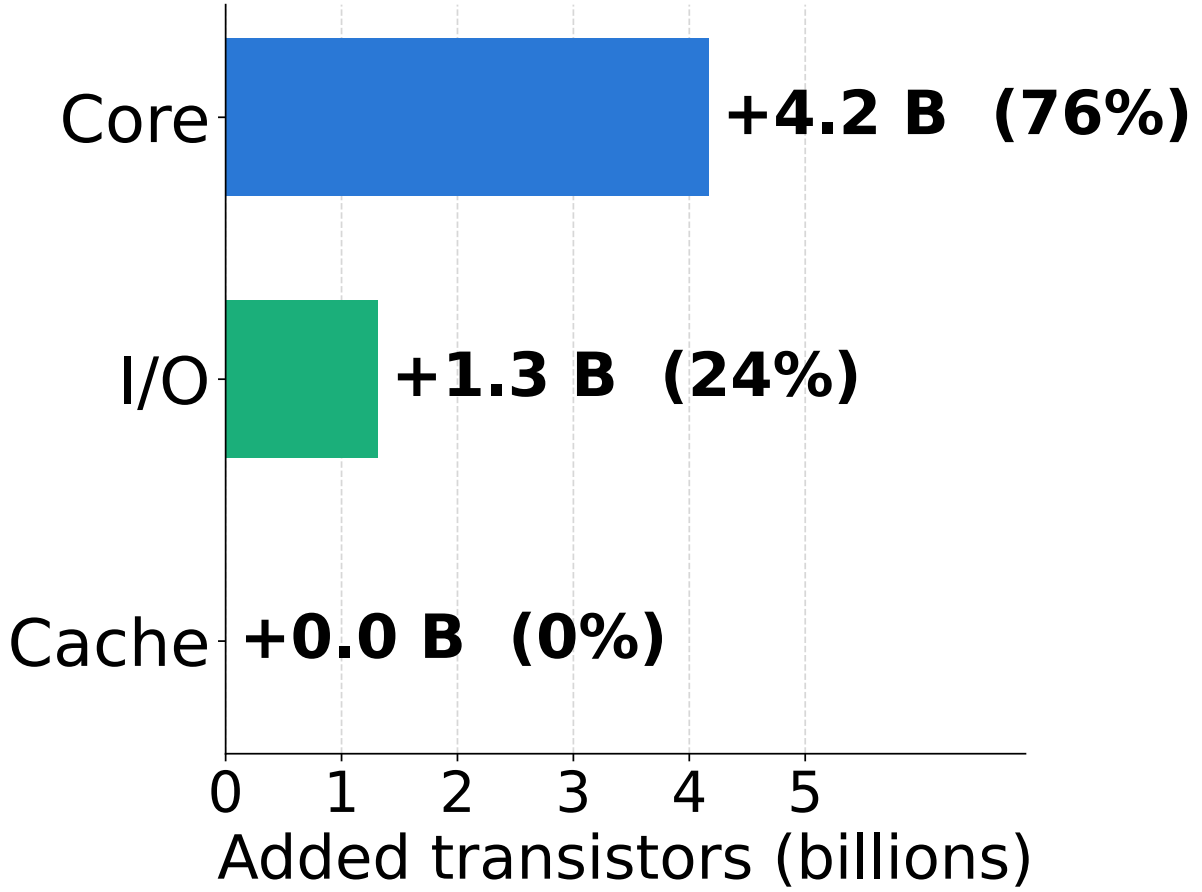
Transistors: AMD Ryzen 7, 2022–2024

- 8 cores
- Up 50% is two years

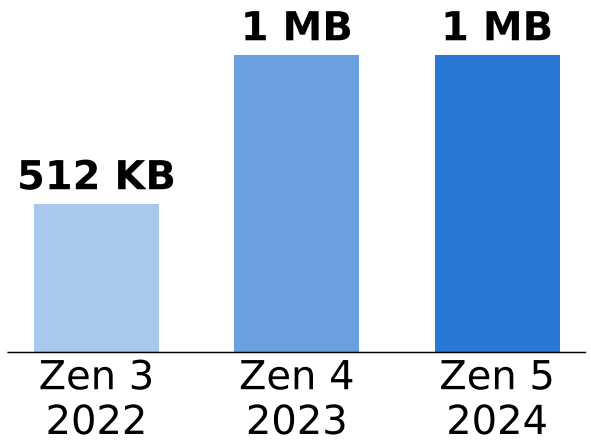


+5.5 B transistors, 2022–2024

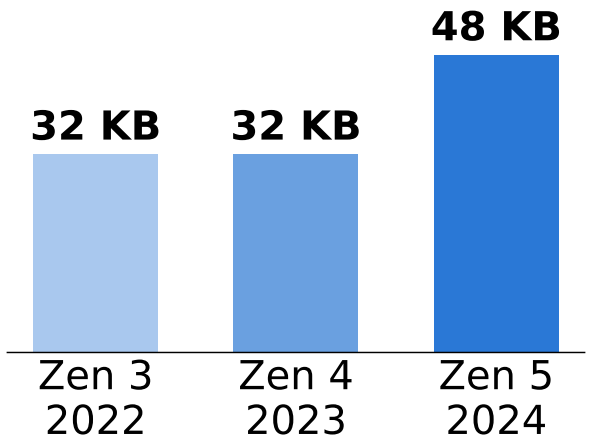
- Same 8 cores
- **76%** went into the core die
- 24% into the I/O die
- Cache die unchanged



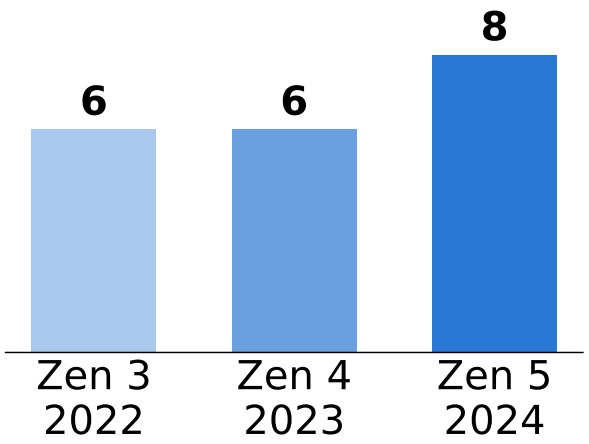
L2 cache per core



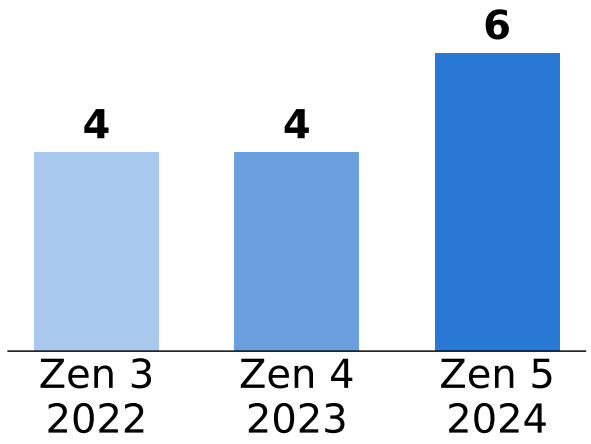
L1 data cache



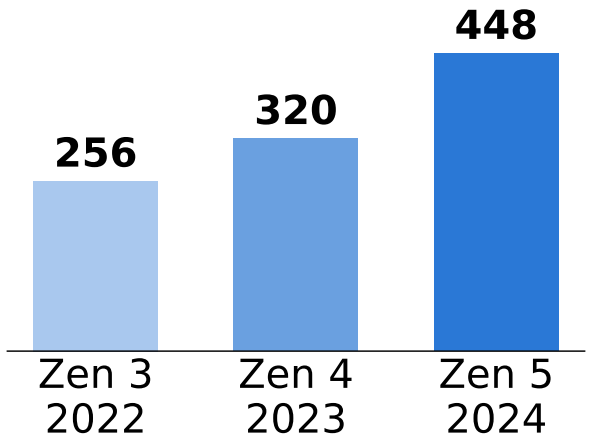
Dispatch width



Integer ALUs

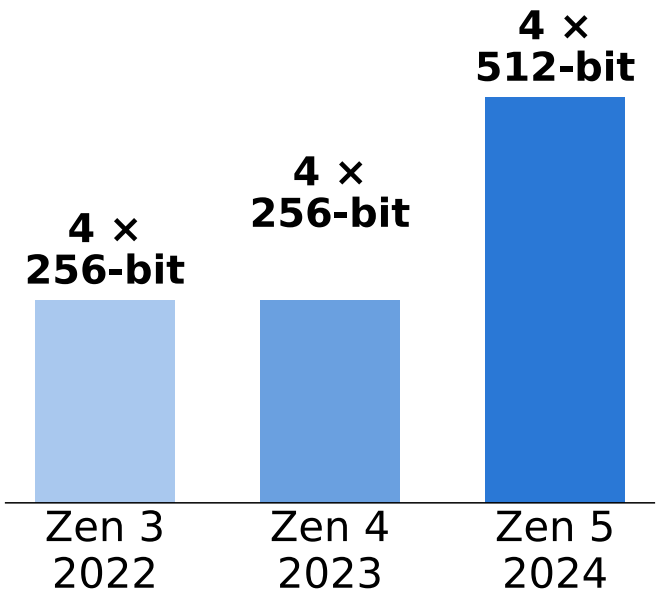


Reorder buffer

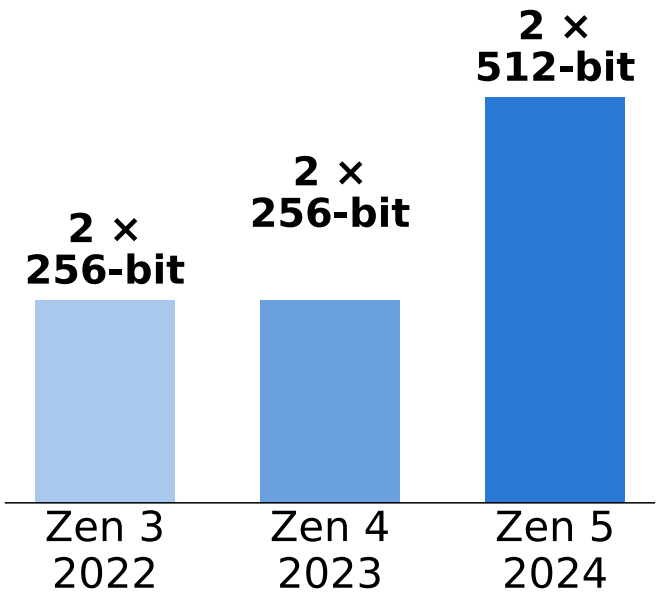


What changed in the core: SIMD (same 8 cores)

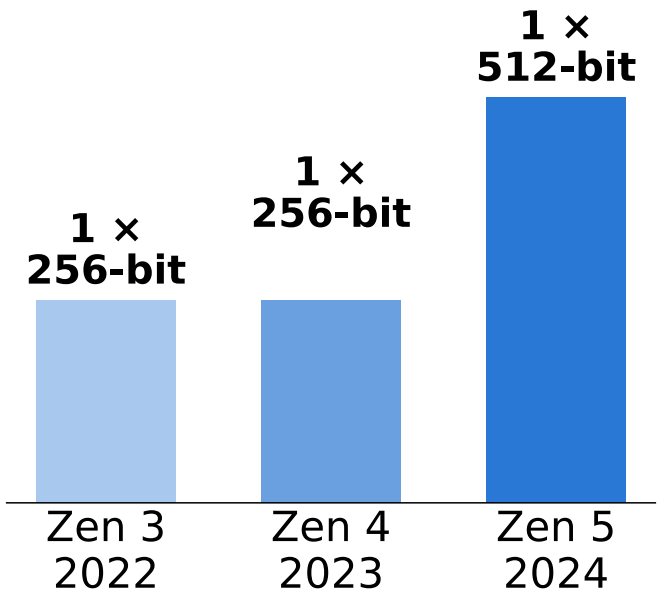
SIMD arithmetic units



Loads per cycle

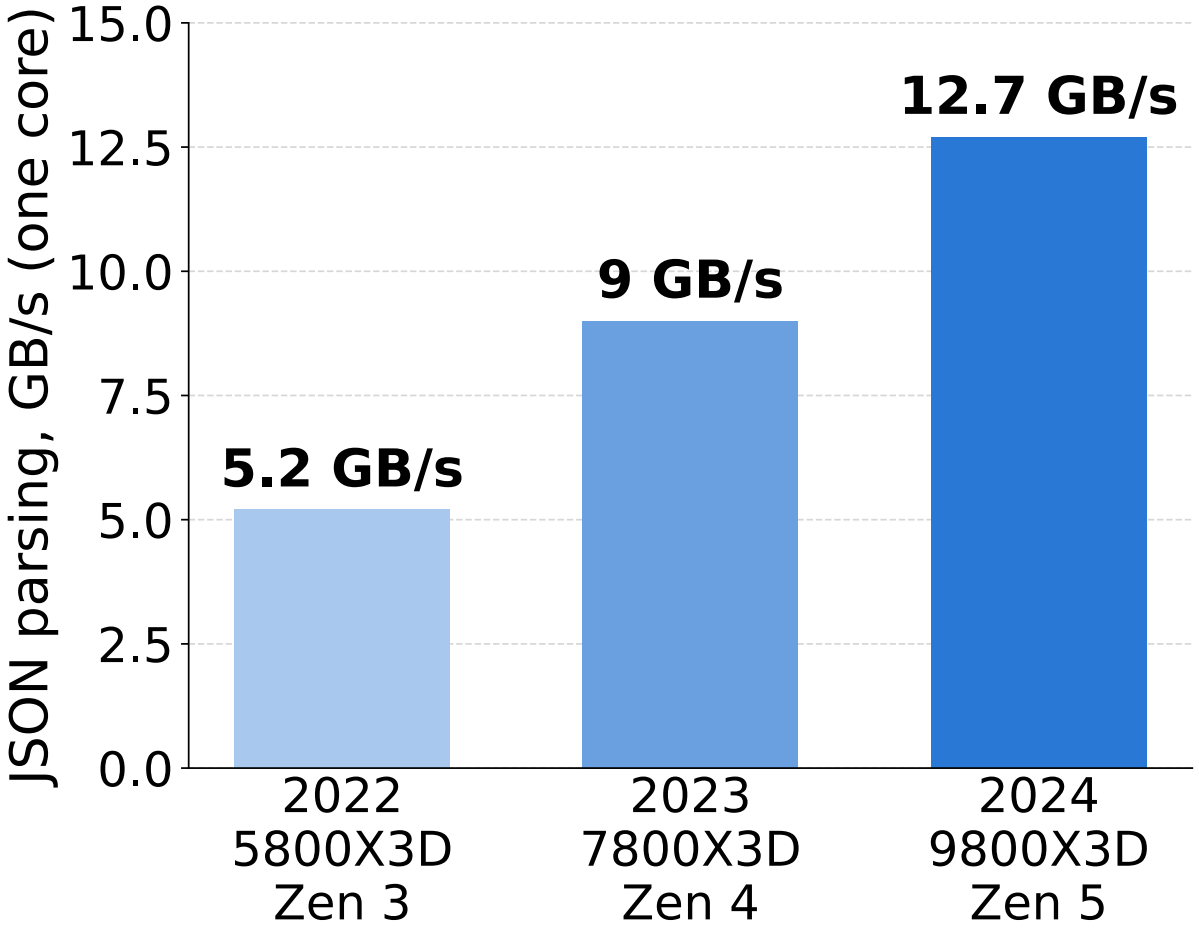


Stores per cycle



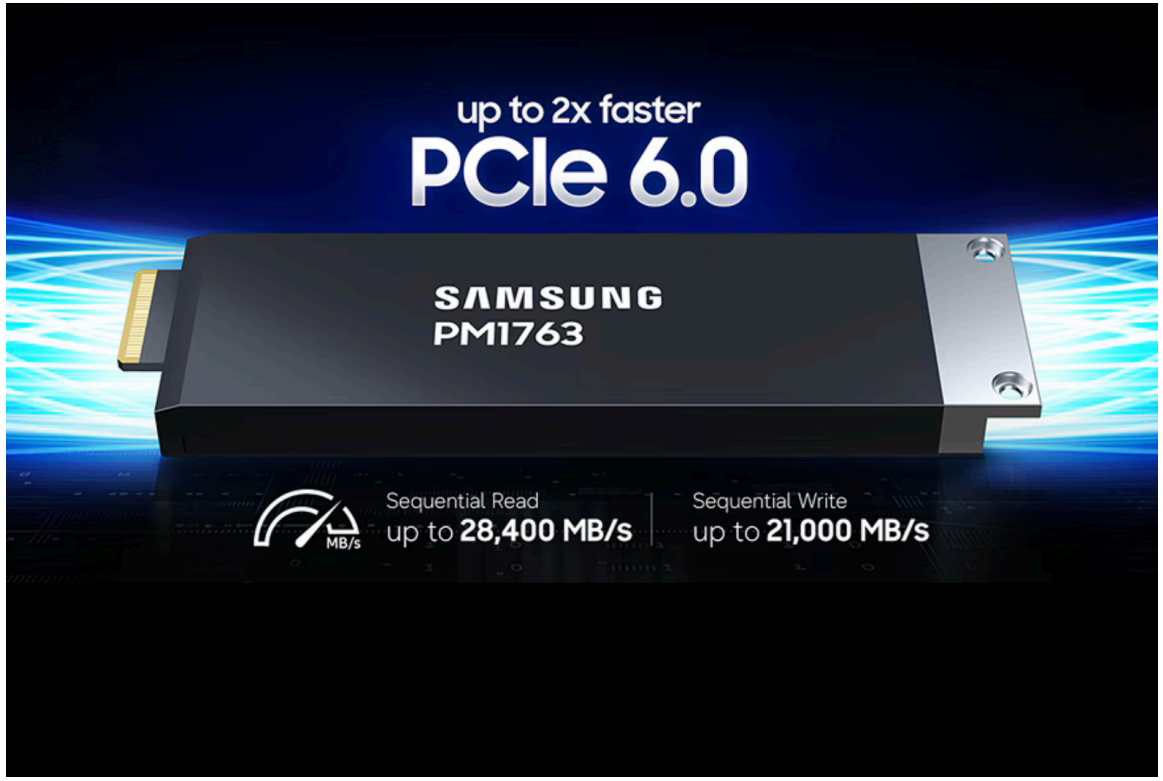
Same software, three years apart

- simdjson, PartialTweets benchmark, one core
- Same code: **2.4×** from 2022 to 2024
- The library is designed for data parallelism
- Same $



Disk at gigabytes per second

device	year	PCI	read speed
PS5	2020	4.0	5 GB/s
WD Black SN8100	2025	5.0	14.9 GB/s
Samsung PM1763	2026	6.0	28.4 GB/s

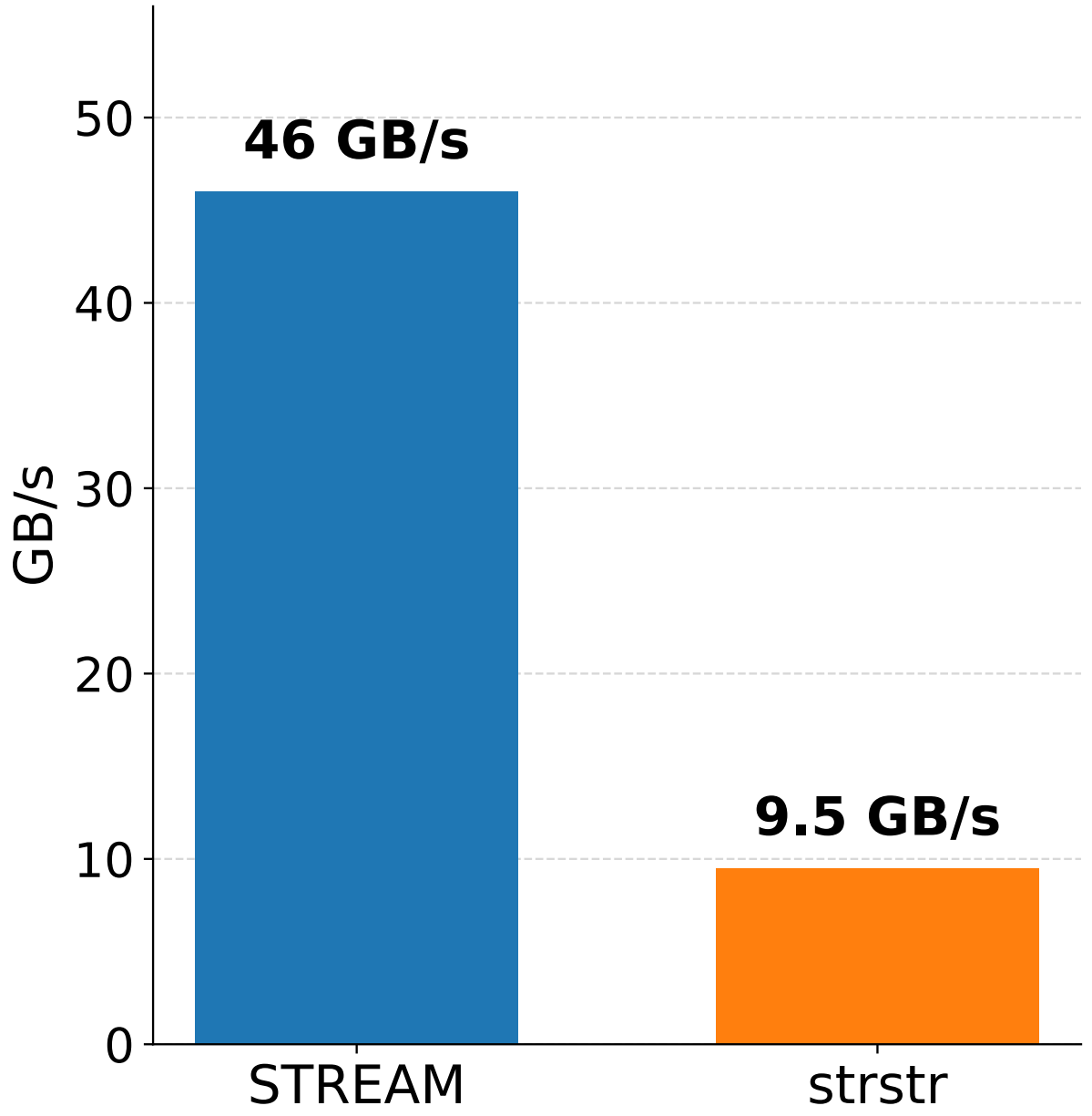


You are CPU bound.

More than you know.

Zen 5 AWS (EPYC 9R45, c8a)

- STREAM single thread bandwidth: 46 GB/s
- `strstr`, 32-byte needle: 9.5 GB/s

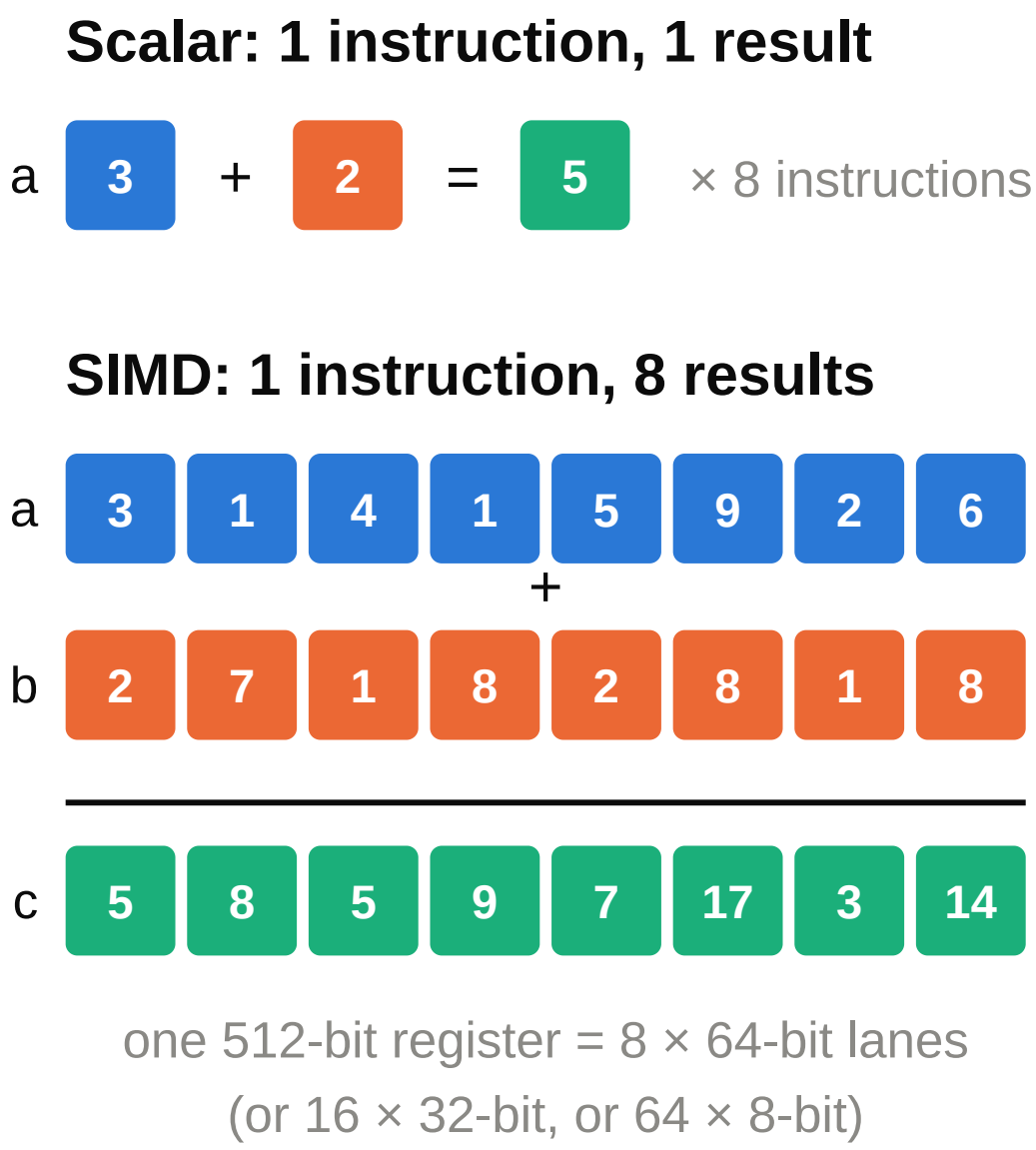


Data-level parallelism

The rest of this talk

SIMD (Single Instruction, Multiple Data)

- Process 16, 32 or 64 bytes with **one** instruction
- Supported on every modern CPU — your phone included
- Not a niche feature



The instruction sets

- **x64**: SSE2 (baseline, 128-bit), AVX2 (256-bit), AVX-512 (512-bit)
- **ARM**: NEON (128-bit, baseline on ARMv8), SVE/SVE2 (scalable)
- **RISC-V**: RVV (scalable)
- **POWER**: AltiVec/VSX · **LoongArch**: LSX, LASX
- **WebAssembly**: 128-bit SIMD, in every browser

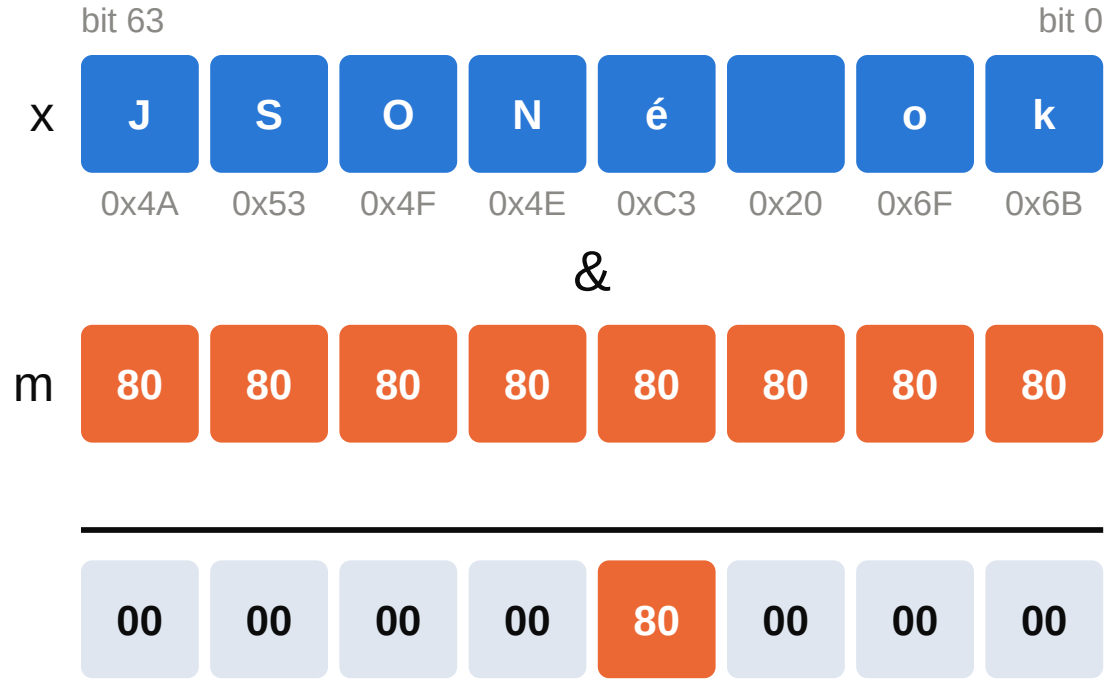
Portability: compile several kernels, dispatch at runtime on CPU features. C++26 adds data-parallel types (`std::simd`).

SWAR: SIMD within a register

- Use plain 64-bit integer instructions
- A 64-bit register holds 8 bytes
- Fully portable C, C++, Rust, Go, Java...
- No intrinsics, no dispatch, no `#ifdef`
- Requires some cleverness

A great place to start.

Is every byte ASCII?



non-zero → a non-ASCII byte

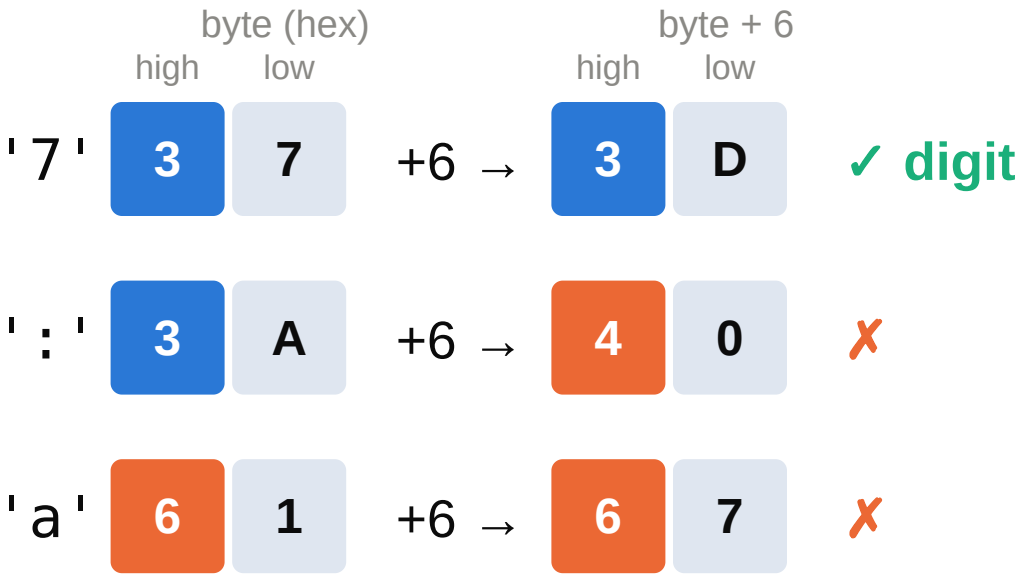
```
m = 0x8080808080808080;
ascii = ((x & m) == 0);
```

8 bytes checked with one 64-bit AND

Check whether we have 8 digits

```
add     rax, rdi
add     rdi, rdx
or      rax, rdi
test    rax,
```

Is this byte a digit?



Digit ⇔ high nibble is **3** before *and* after +6

0x30 … 0x39 → 0x36 … 0x3F

0x3A … 0x3F → 0x40 … 0x45 (carries out)

Same trick on 8 bytes at once: next slide

Case study: ASCII processing

ASCII to lower case

```
For each character c
    If c - 'A' <= 'Z' - 'A' then
        c = c + 'a' - 'A'
    EndIf
EndFor
```

One byte per iteration. One unpredictable branch per byte.

64 characters in 3 instructions

input c

```
A = set1('A'), Z_A = set1(25), a_A = set1(32)
```



↓

1. subtract 'A'

```
ca = _mm512_sub_epi8(c, A)
```

wraps around when c < 'A'



↓

2. compare ≤ 25, unsigned

```
mask = _mm512_cmple_epu8_mask(ca, Z_A)
```

one compare checks both bounds



↓

3. add 32 where mask is set

```
out = _mm512_mask_add_epi8(c, mask, c, a_A)
```

other lanes keep c



3 instructions, 64 characters, no branch

Let us check with llvm-mca

- `llvm-mca` is a static machine-code analyzer shipped with LLVM
- Feed it assembly, name a microarchitecture, get a cycle estimate
- No hardware required, no noise, no warm-up

```
llvm-mca -mcpu=icelake-server -iterations=100 kernel.s
```

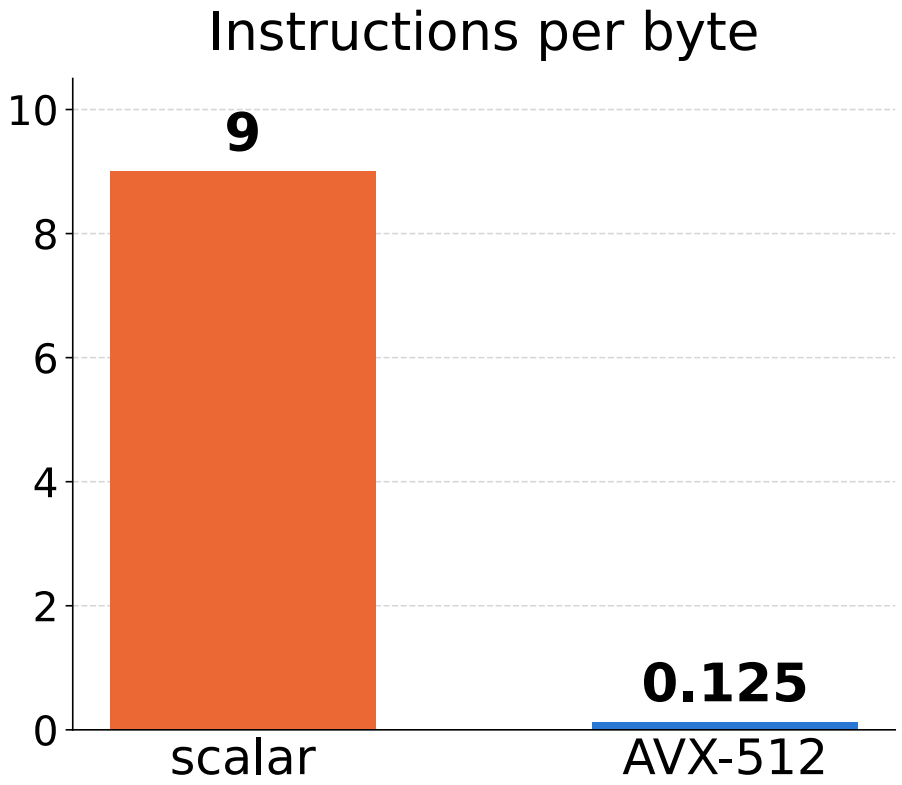
The scalar loop: 1 byte per iteration

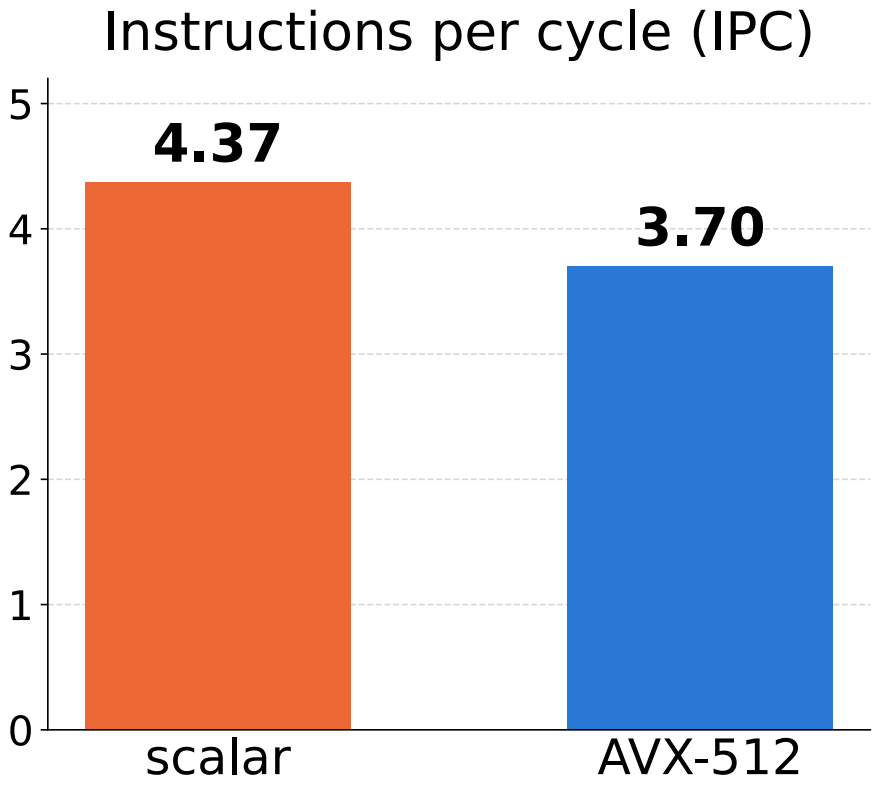
```
movzbl  (%rdi,%rax), %ecx
leal    -65(%rcx), %edx
cmpb    $26, %dl
jae     .LBB1_2
addb    $32, %cl
movb    %cl, (%rdi,%rax)
addq    $1, %rax
cmpq    %rax, %rsi
jne     .LBB1_1
```

The AVX-512 loop: 64 bytes per iteration

```
vmovdqu64  (%rdi,%rax), %zmm2
vpsubb     %zmm0, %zmm2, %zmm3
vpcmpub    $2, %zmm1, %zmm3, %k1
vpaddb     %zmm4, %zmm2, %zmm2 {%k1}
vmovdqu64  %zmm2, (%rdi,%rax)
addq       $64, %rax
cmpq       %rax, %rsi
jne        .LBB0_1
```

llvm-mca says...





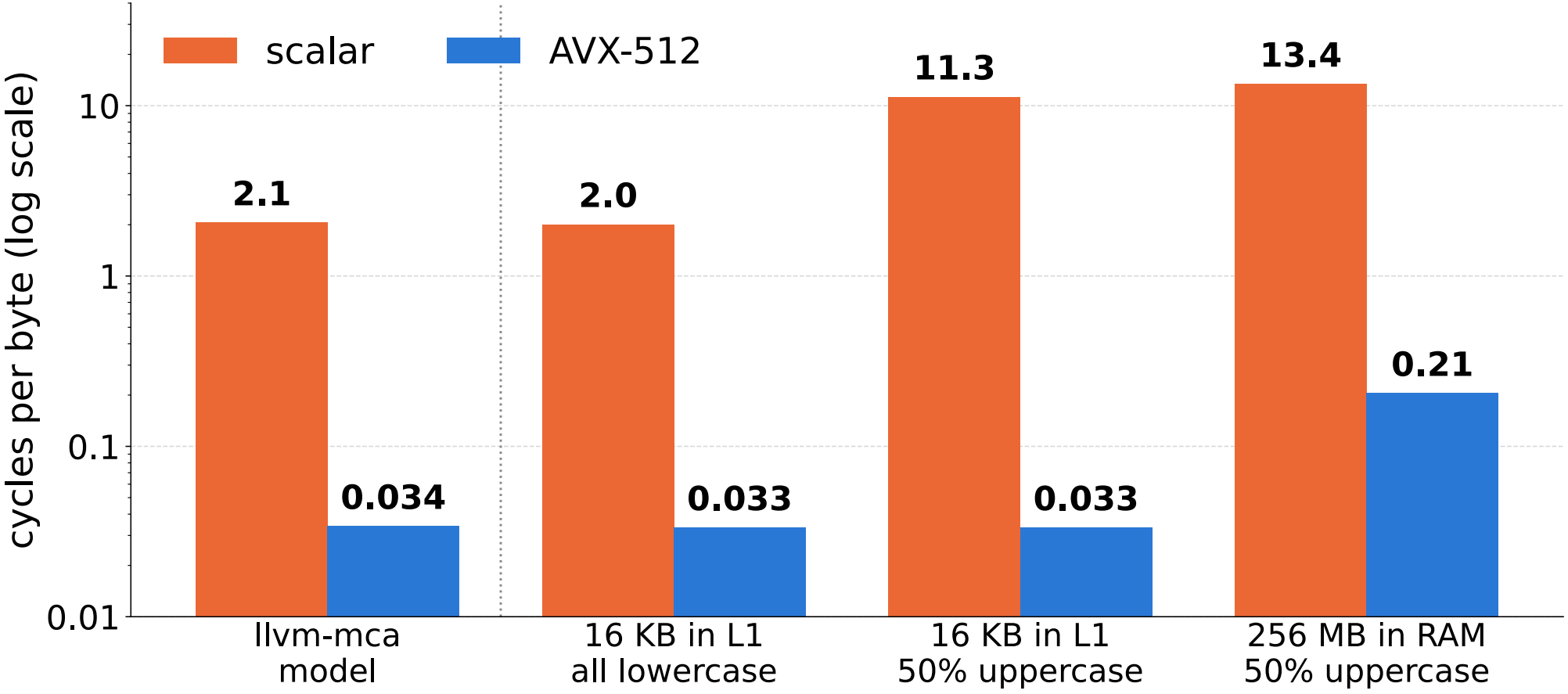
About 60× fewer cycles per byte.

What llvm-mca will not tell you

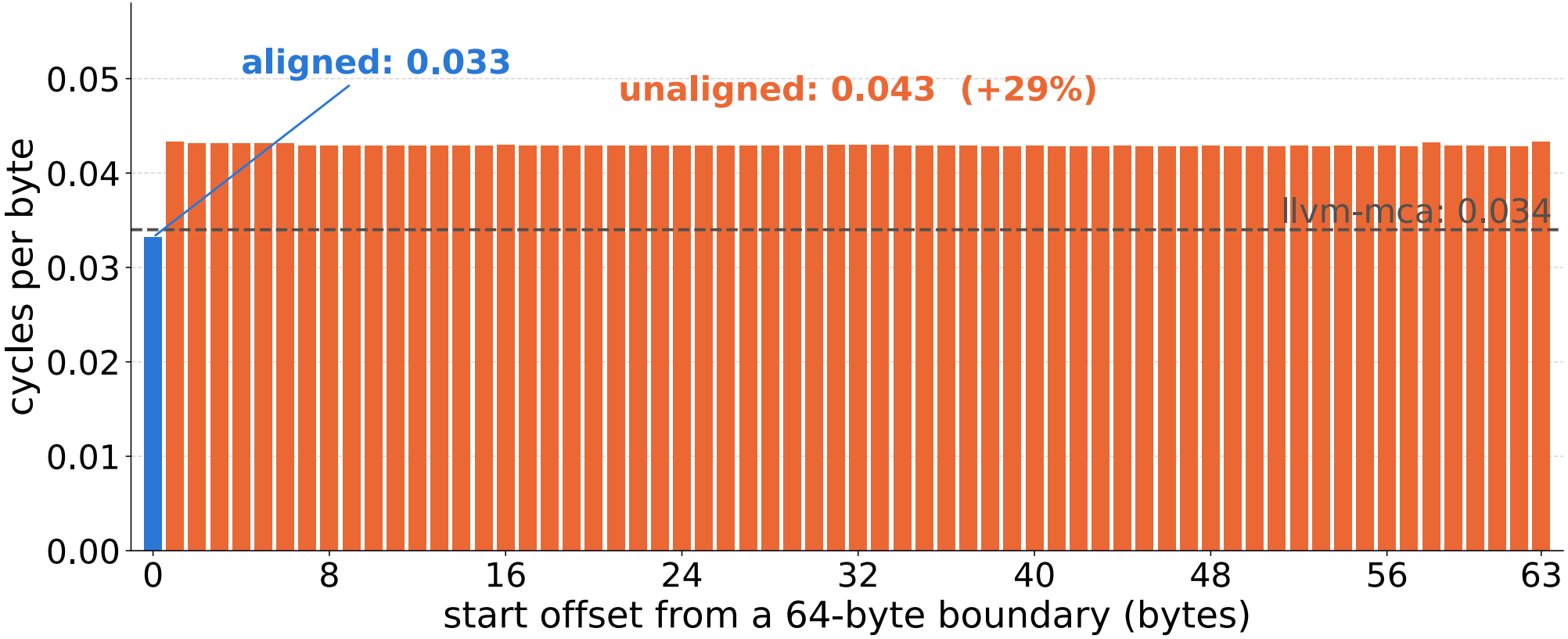
- It assumes perfect branch prediction
- It assumes every load hits L1
- It often ignores memory alignment issues
- It models one microarchitecture at a time.

It is a model. Not reality.

Reality: Intel Xeon Gold 6548N (Emerald Rapids)



Alignment: 64-byte loads and stores, 64 offsets



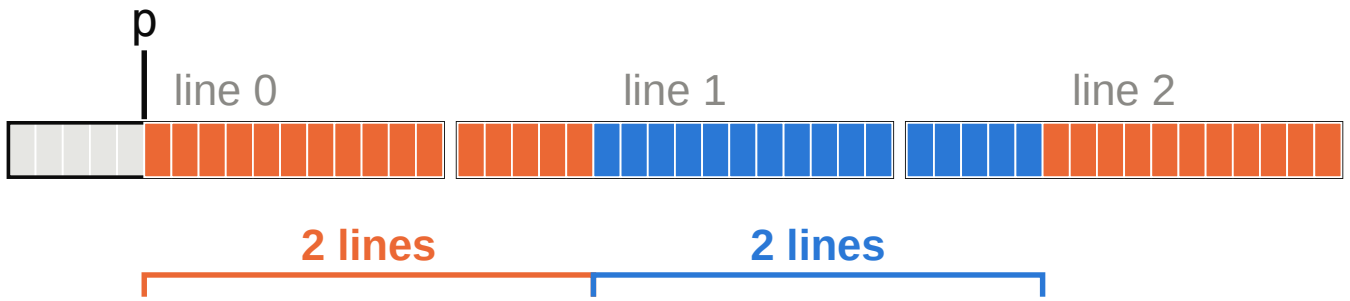
Only 1 offset in 64 matches the model. The other 63 straddle two cache lines: **+30%**.

Masked loads and stores

- AVX-512 loads and stores take a 64-bit mask: one bit per byte
- Masked-out bytes are not read, not written, and **cannot fault**
- Round `p` down to the cache line, mask off the bytes before `p`: the rest of the loop is aligned
- Same trick handles the tail: no scalar loop, no branch

Naive

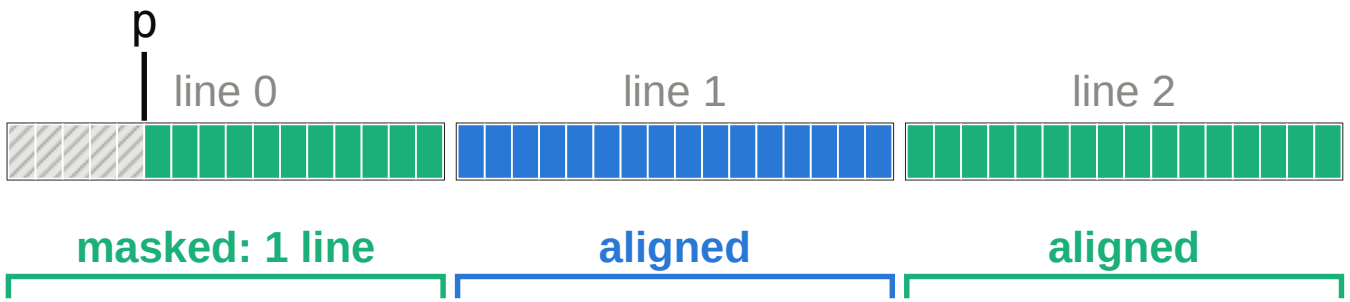
64-byte loads at p



every access straddles 2 lines

Masked head

```
p0 = p & ~63
m = ~0 << (p & 63)
```



mask bit 0: not read, not written, cannot fault

```
_mm512_maskz_loadu_epi8(m, p0) _mm512_mask_storeu_epi8(p0, m, v)
```

Case study: JSON

Joint work with many people such as Geoff Langdale (Intel), John Keiser (Microsoft), Francisco Geiman Thiesen (Microsoft), etc.

JSON

- Portable, simple, human-readable; used by ~97% of API requests
- Strings (escaped), numbers, objects, arrays
- Reading and writing JSON is often *slow*: 100 MB/s to 300 MB/s
- Slower than a fast disk or a fast network

```
$ go run parse_twitter.go
Parsed 0.63 GB in 6.961 seconds (90.72 MB/s)
```

You are probably using simdjson

- Node.js, Electron
- ClickHouse
- WatermelonDB, Apache Doris, Meta Velox, Milvus, QuestDB, StarRocks






```
{
    "nom": "Marie Dubois",
    "age": 28,
    "ville": "Paris",
    "loisirs": ["lecture", "voyage", "yoga"],
    "contact": {
        "email": "marie.dubois@example.com",
        "phone": "+33123456789"
    }
}
```

Every major JavaScript engine parses JSON with SIMD

- `JSON.parse` in Node.js, Bun and Deno is data-parallel (simdjson inspired).
- Billions of calls per second worldwide.
- Nobody had to change a single line of JavaScript.

simdjson: two-stage design

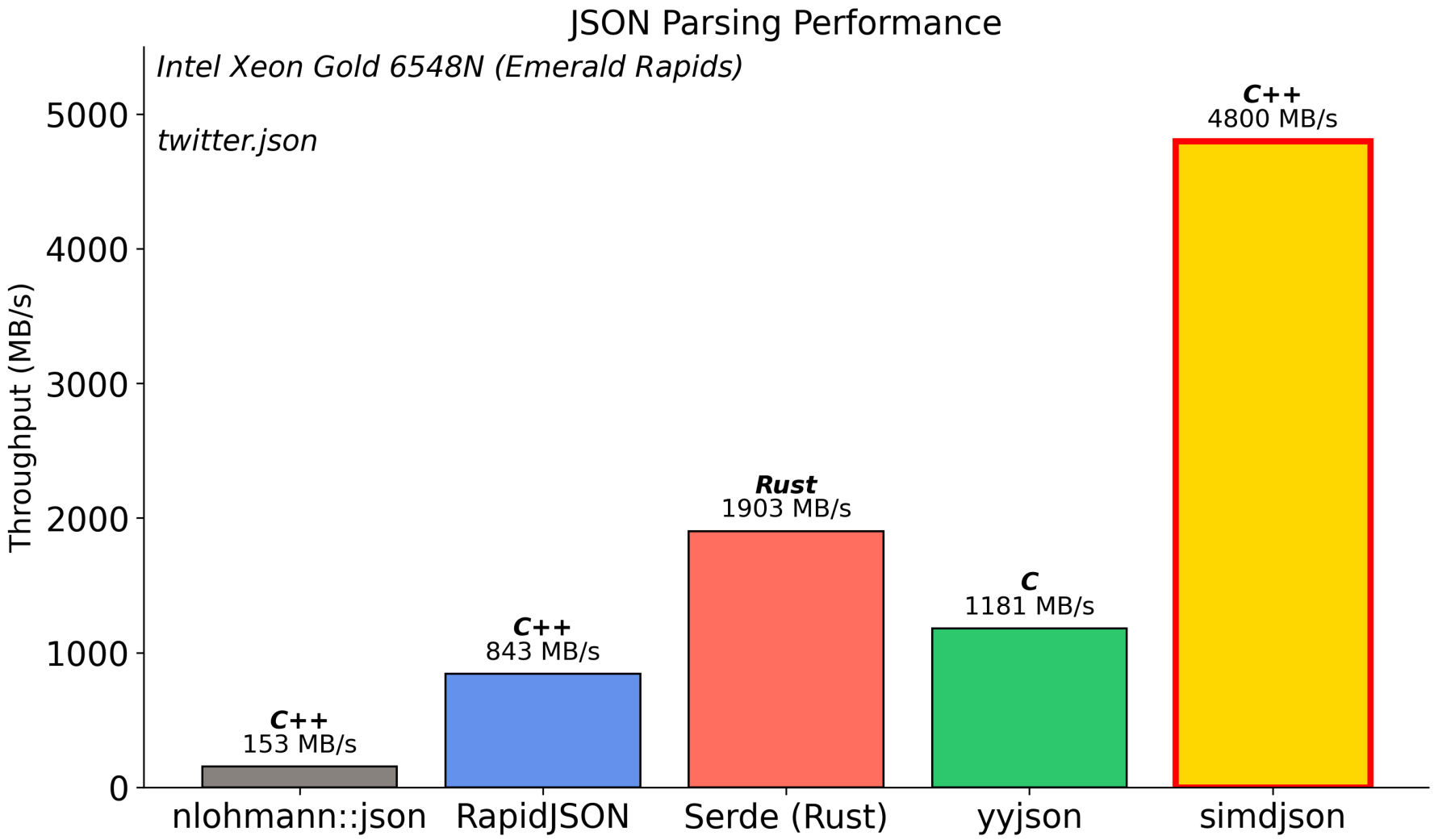
Stage 1 (data-parallel): scan the whole document with SIMD

- find every structural character and the start of every string
- validate UTF-8
- produce an index

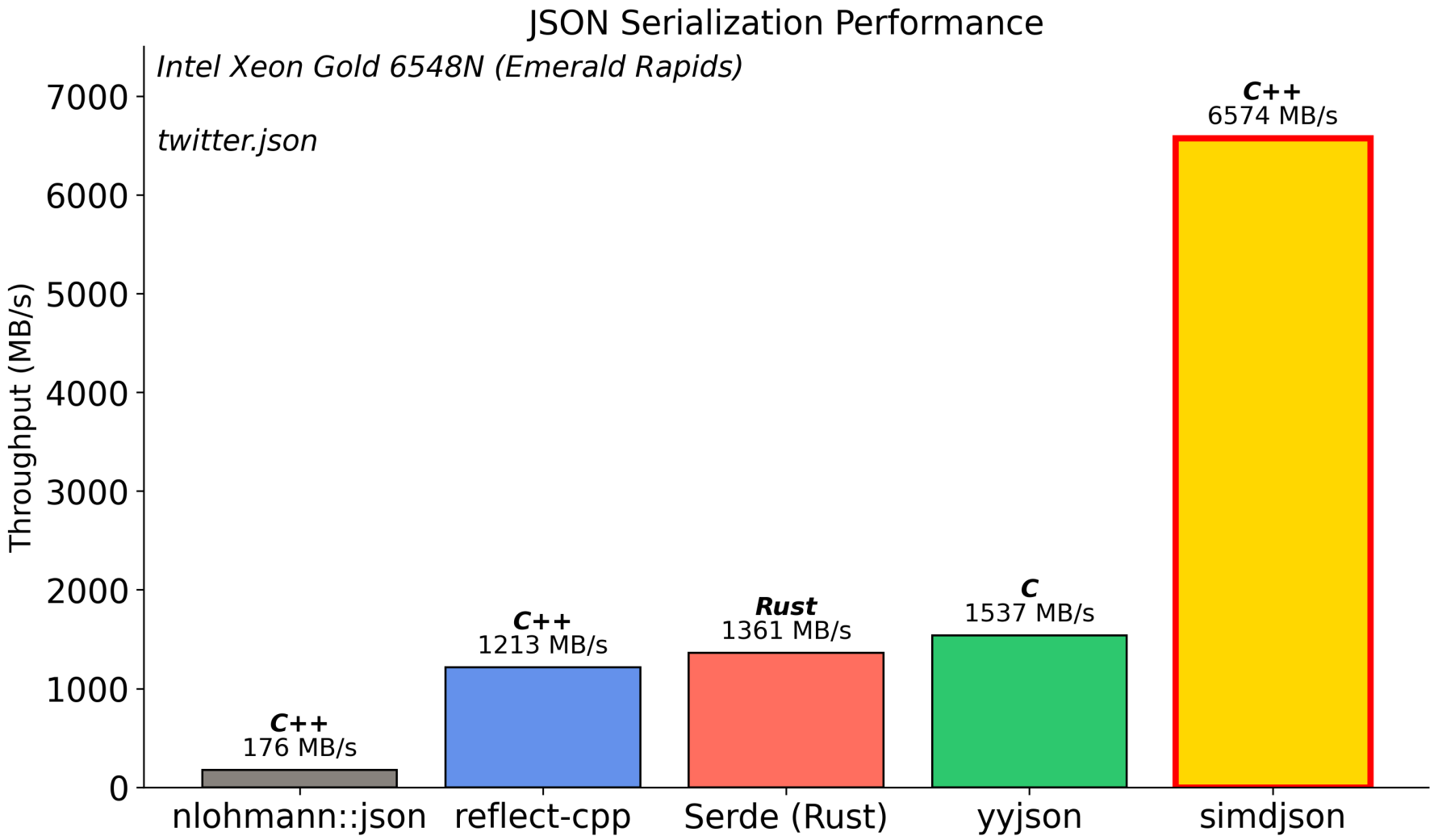
Stage 2 (mostly serial): walk the index and build values

- the hard, branchy work now runs on ~5% of the bytes

Deserialization (Intel Xeon Gold 6548N)



Serialization (Intel Xeon Gold 6548N)

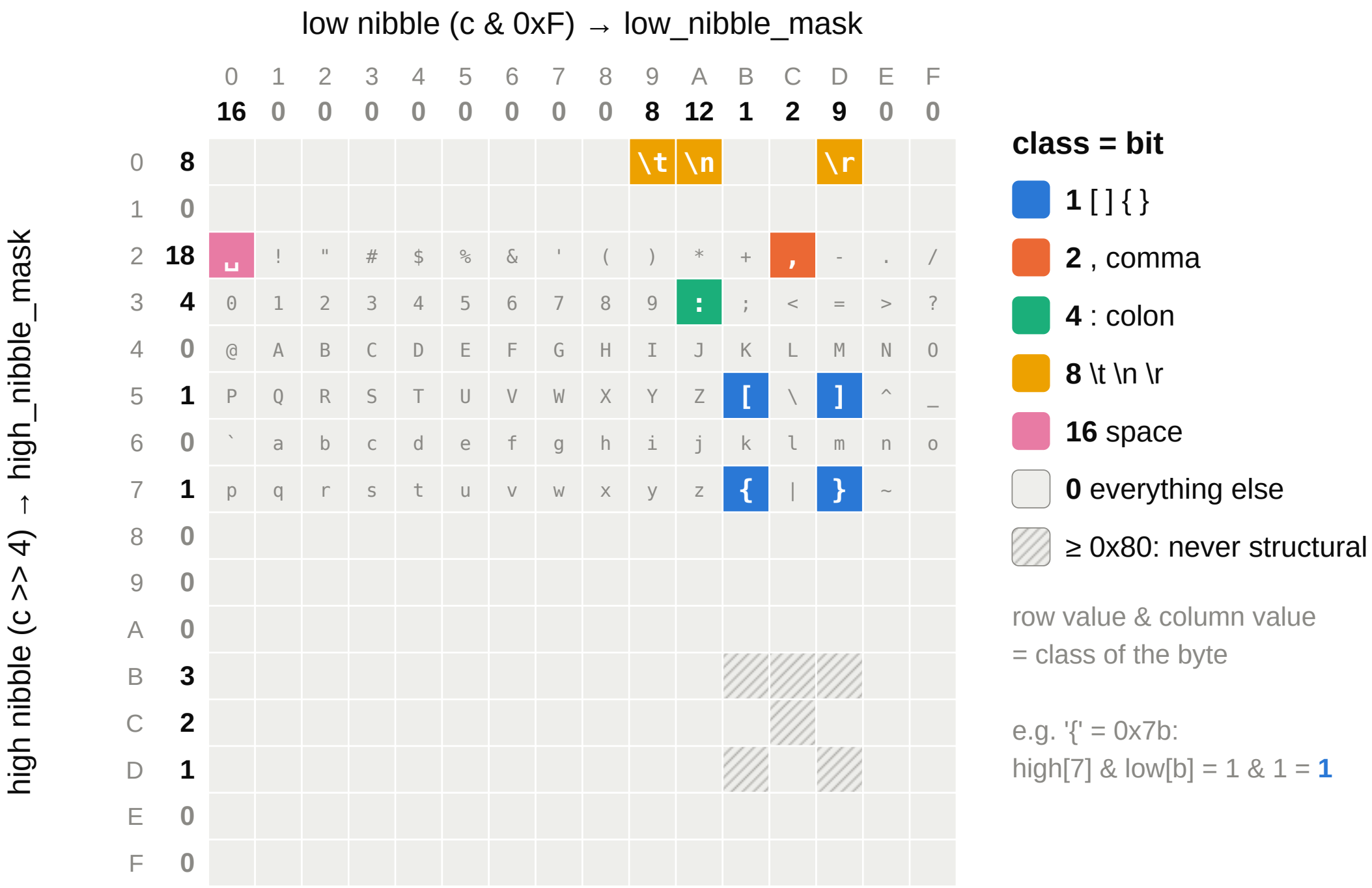


Classifying characters

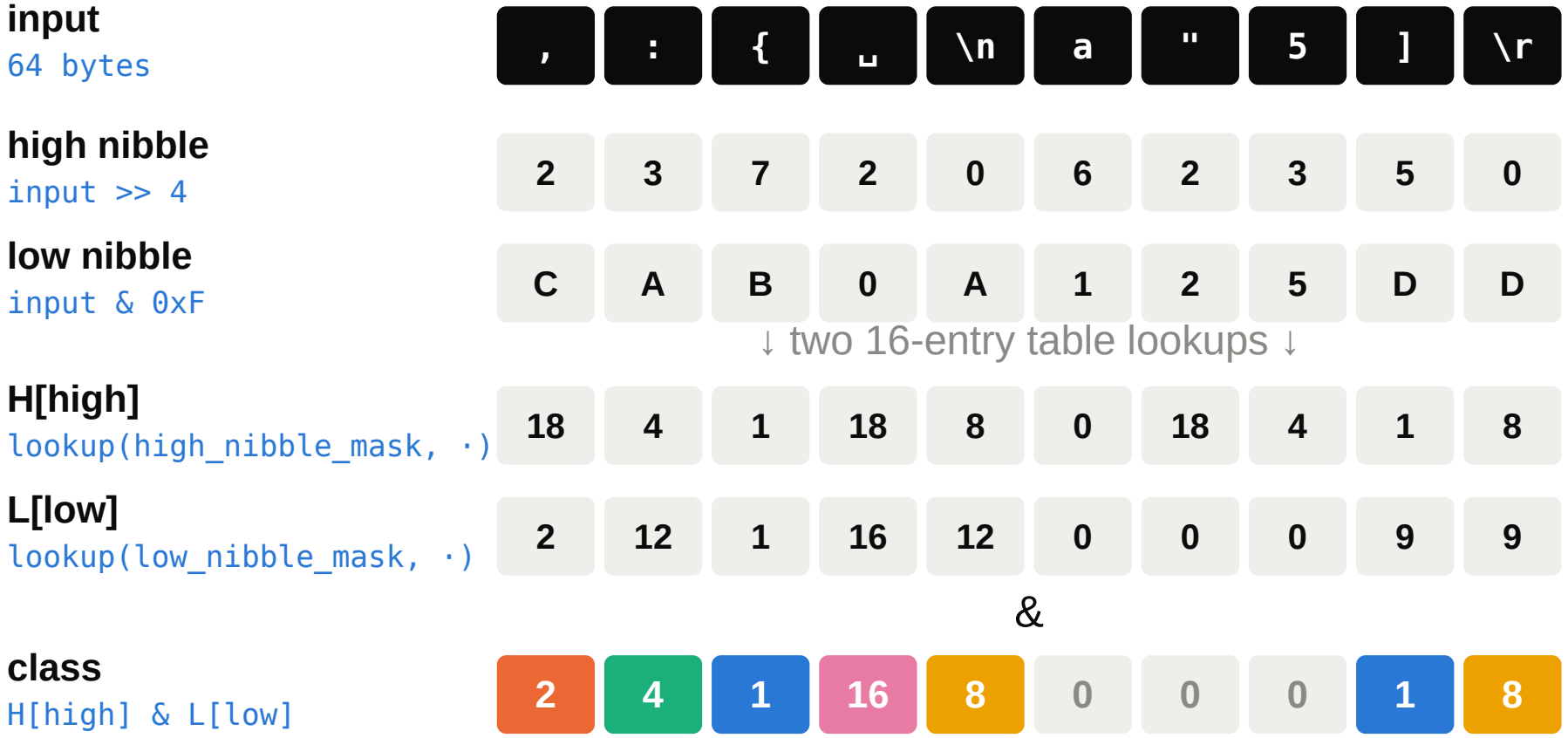
We need to sort every byte into a class:

- comma (0x2c) `,`
- colon (0x3a) `:`
- brackets (0x5b, 0x5d, 0x7b, 0x7d): `[, ], {, }`
- white-space (0x09, 0x0a, 0x0d, 0x20)
- everything else

A switch statement per byte? No.

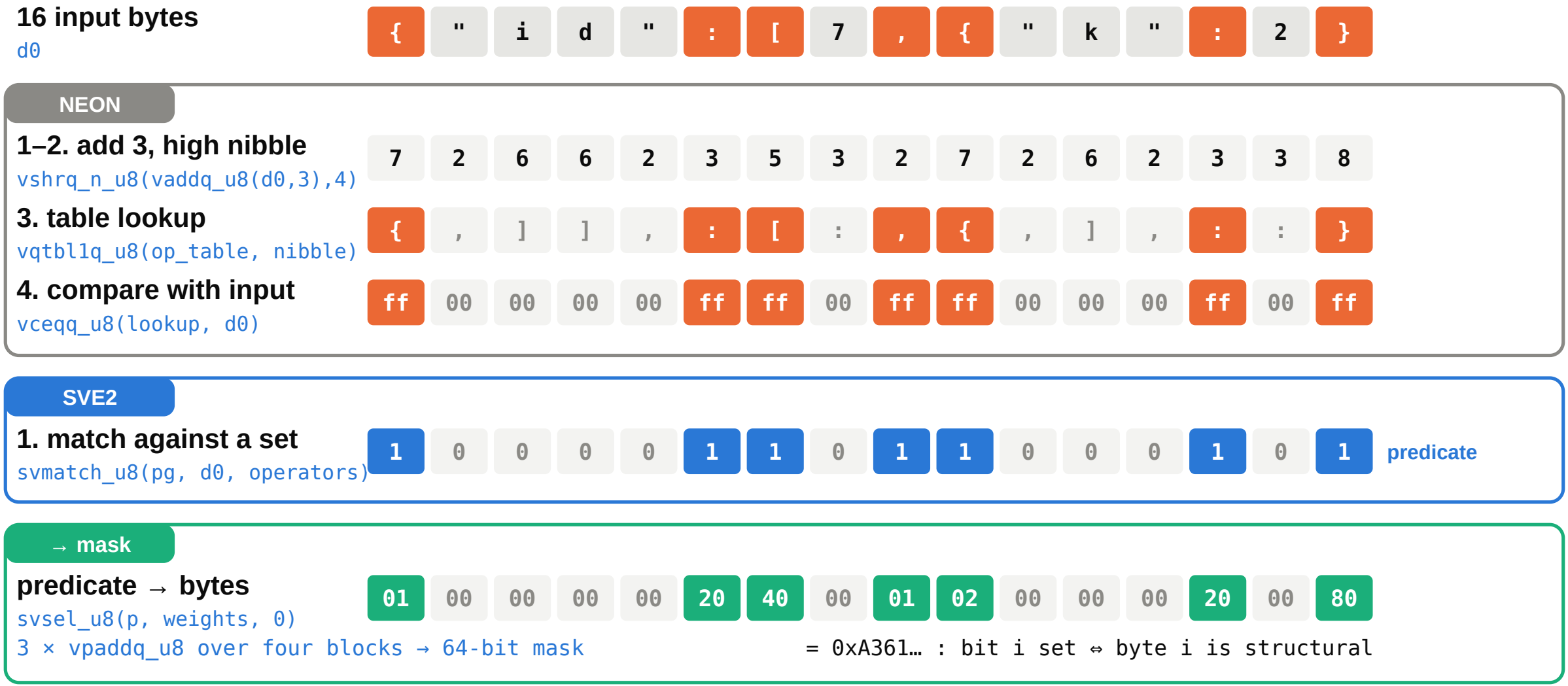

low nibble (c & 0xF) → low_nibble_mask
high nibble (c >> 4) → high_nibble_mask
0 1 2 3 4 5 6 7 8 9 A B C D E F
16 0 0 0 0 0 0 0 0 8 12 1 2 9 0 0
0 8 \t \n \r
1 0
2 18 ␣ ! " # $ % & ' ( ) * + , - . /
3 4 0 1 2 3 4 5 6 7 8 9 : ; < = > ?
4 0 @ A B C D E F G H I J K L M N O
5 1 P Q R S T U V W X Y Z [\ ] ^ _
6 0 ` a b c d e f g h i j k l m n o
7 1 p q r s t u v w x y z { | } ~
8 0
9 0
A 0
B 3
C 2
D 1
E 0
F 0
class = bit
1 [ ] { }
2 , comma
4 : colon
8 \t \n \r
16 space
0 everything else
≥ 0x80: never structural
row value & column value
= class of the byte
e.g. '{' = 0x7b:
high[7] & low[b] = 1 & 1 = 1

Classification, 64 bytes at a time



5 instructions per 64 bytes; 1 = bracket, 2 = comma, 4 = colon, 8 = \t\n\r, 16 = space, 0 = other

NEON vs SVE2 `match`, one 16-byte block



Serialization is also a data-parallel problem

- JSON requires escaping `"`, `\`, and control characters.
- Almost no string actually needs escaping.
- So: check 16 to 64 bytes at once, and take the fast path.

SIMD string escaping

32 bytes of a string

A	l	m	o	s	t		n	o		s	t	r	i	n	g		n	e	e	d	s		e	s	c	a	p	i	n	g	!
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32

1 byte at a time

```
for (char c : str)
if (c == '"' || c == '\\'
|| c < 0x20) ...
```

32 iterations, each: 3 compares + 1 data-dependent branch. The loop waits for each branch before the next byte.

32 bytes at once

1. quote '"'

```
vector == '"'
```

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	mask

2. backslash '\'

```
vector == '\\'
```

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	mask

3. control characters

```
vector < 0x20
```

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	mask

4. OR the three masks

```
needs_escape = m1 | m2 | m3
```

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	= 0

```
3 compares + 1 branch
per 32 bytes
```

5. one branch: mask == 0 → copy the 32 bytes verbatim (the fast path)

mask ≠ 0 (rare) → the first set bit points at the byte to escape

Case study: IP addresses

Joint work with Yagiz Nizipli (SpaceX)

`192.168.0.1` → 32 bits

- Every server logs the client address. Every firewall evaluates a rule. Every URL parser meets an IP literal.
- The usual tool: `inet_pton` from the C library, ~340 instructions per address.
- An IPv4 address is at most 15 bytes: it fits in **one** 16-byte register.
- An IPv6 address is at most 45 bytes: it fits in **one** 64-byte register.

Load the whole address once. Never walk it.

IPv4 in six steps, no loop

1. masked load, len = 11

`mask_loadu_epi8(pad='0', len)`

pad with '0': neither dot nor error

2. dots → partition p

`p = cmpeq(v, '.') | 1 << len`

p = 0xA88: the whole layout in 16 bits

3. perfect hash → key

`k = (p * 0x00CF7800) >> 24`

81 layouts → 81 distinct keys

4. table shuffle

`shuffle_epi8(v, table[k])`

octet → [0, hundreds, tens, ones]

5. VNNI dot product

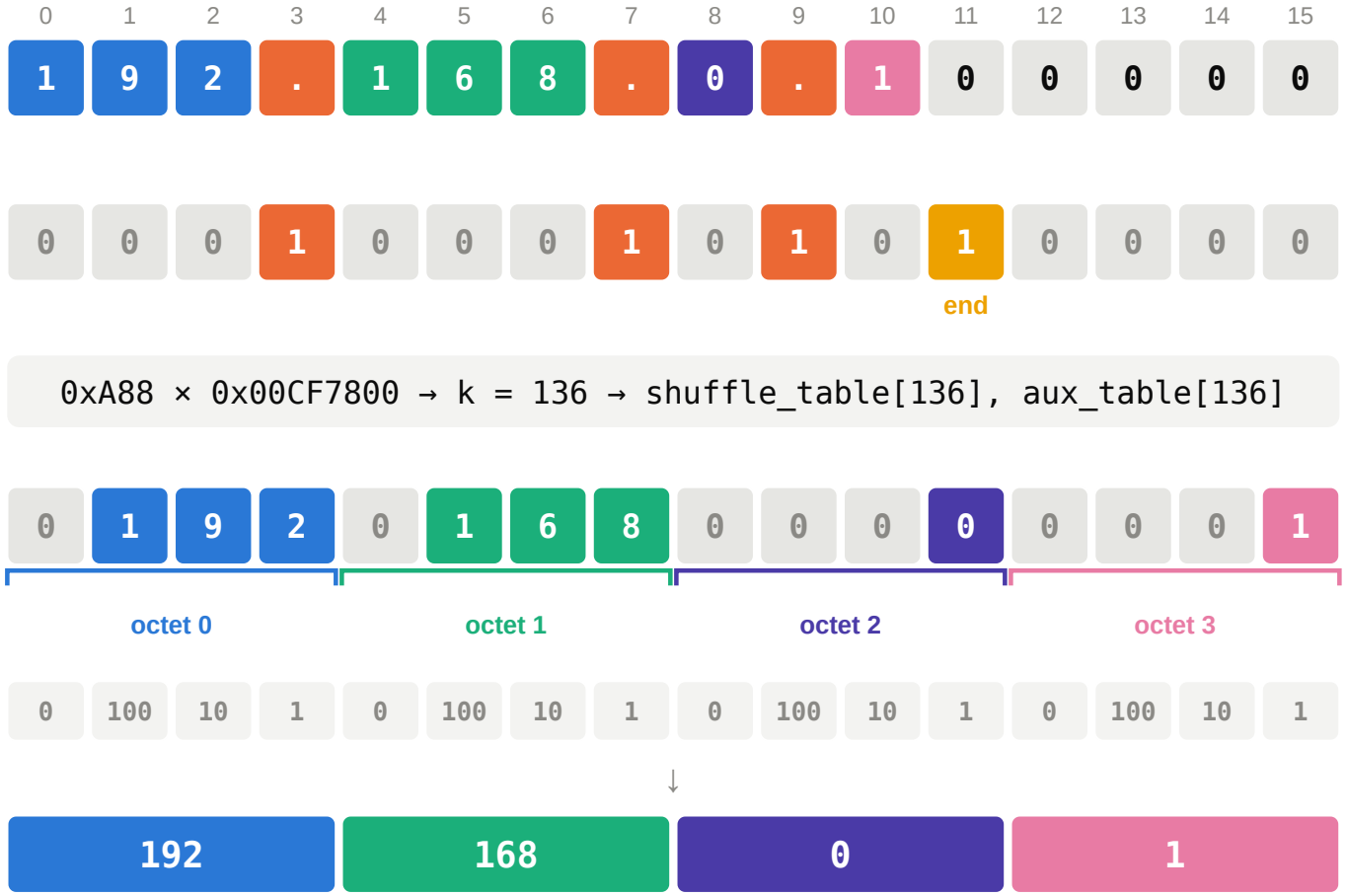
`dpbusd_epi32(digits, weights)`

weights [0, 100, 10, 1] × 4

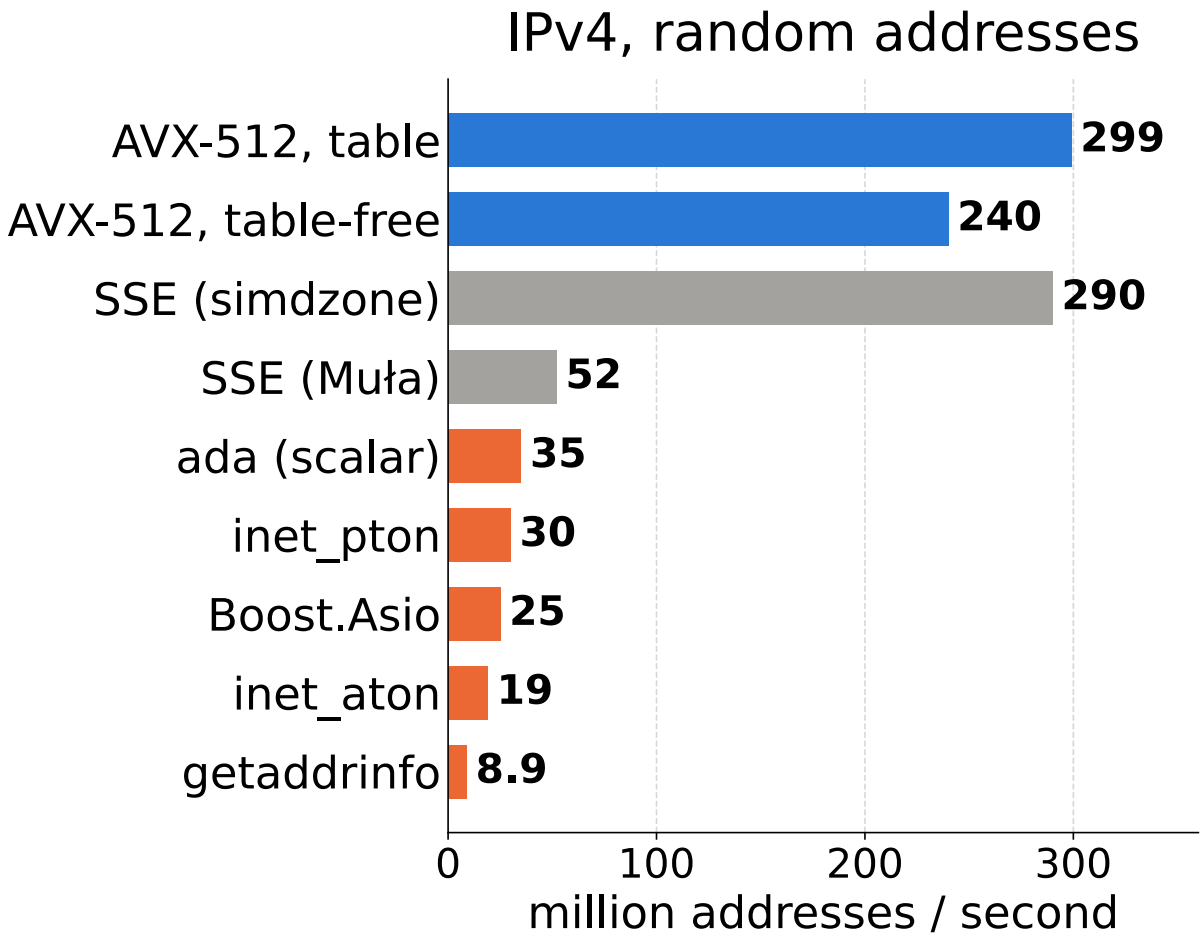
6. validate

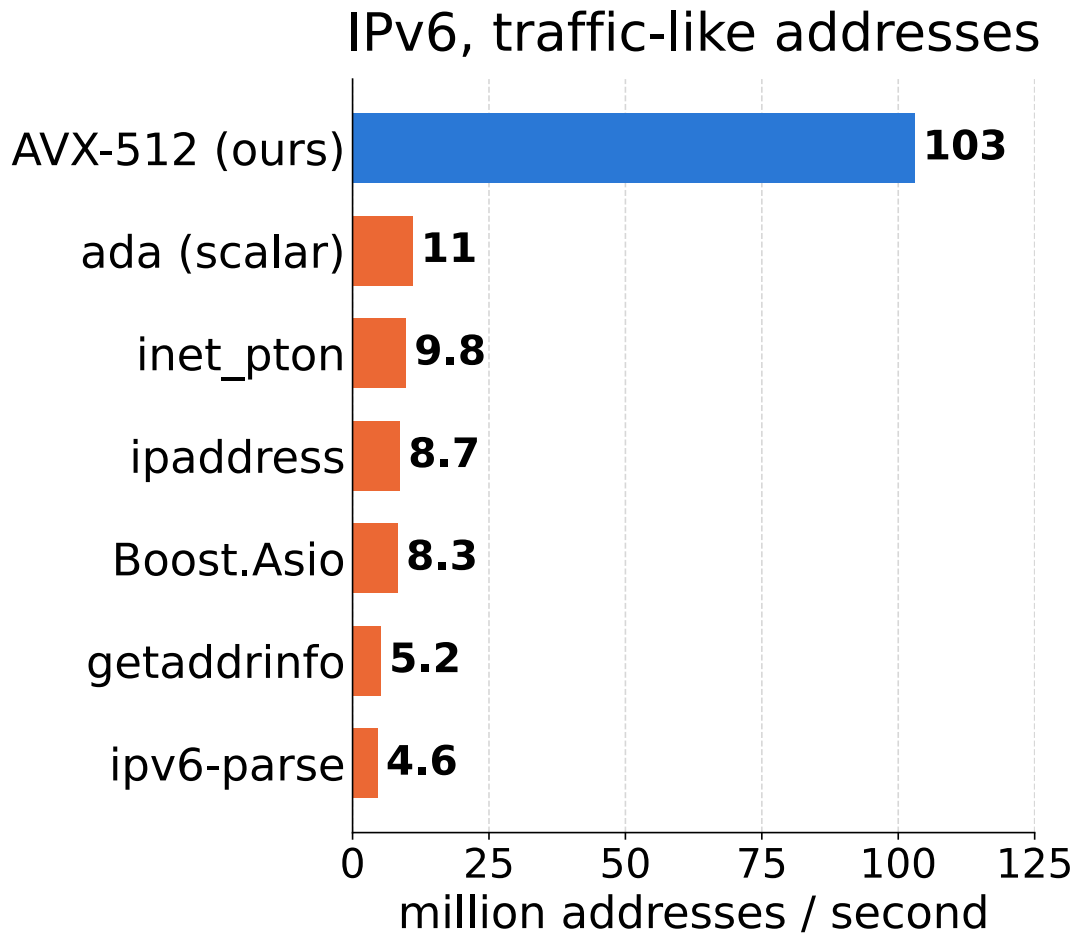
`p == aux[k] | non-digit | > 255`

all in mask registers: one branch



IPv4 parsing, Intel Xeon Gold 6548N





10× over `inet_pton` on IPv4, 10× on traffic-like IPv6. 6–60× fewer instructions.

Part 5

Why your compiler will not do this for you

After decades of autovectorization research

- C# (.NET) has intrinsics
- C++ added std::simd
- Java Vector

Questions?

Daniel Lemire — lemire.me

X: @lemire · GitHub: github.com/lemire



Case study: Unicode

simdutf

- Inside Safari, Chrome, Node.js, Bun
- Unicode transcoding and validation at gigabytes per second
- Base64 too
- x64, ARM, POWER, RISC-V, LoongArch



Unicode (UTF-16)

- Code points from U+0000 to U+FFFF: a single 16-bit value.
- Beyond: a *surrogate pair*, `U+D800` – `U+DBFF` followed by `U+DC00` – `U+DFFF`.
- A lone surrogate makes the string ill-formed.

Every JavaScript string, every Java string, every Windows filename.

Validate

```
PROCEDURE validate_utf16(code_units)
    i ← 0
    WHILE i < |code_units|
        unit ← code_units[i]
        IF unit ≤ 0xD7FF OR unit ≥ 0xE000 THEN
            INCREMENT i
            CONTINUE
        IF unit ≥ 0xD800 AND unit ≤ 0xDBFF THEN
            IF i + 1 ≥ |code_units| THEN
                RETURN false
            next_unit ← code_units[i + 1]
            IF next_unit < 0xDC00 OR next_unit > 0xDFFF THEN
                RETURN false
            i ← i + 2  // Valid surrogate pair
            CONTINUE
        RETURN false
    RETURN true
```

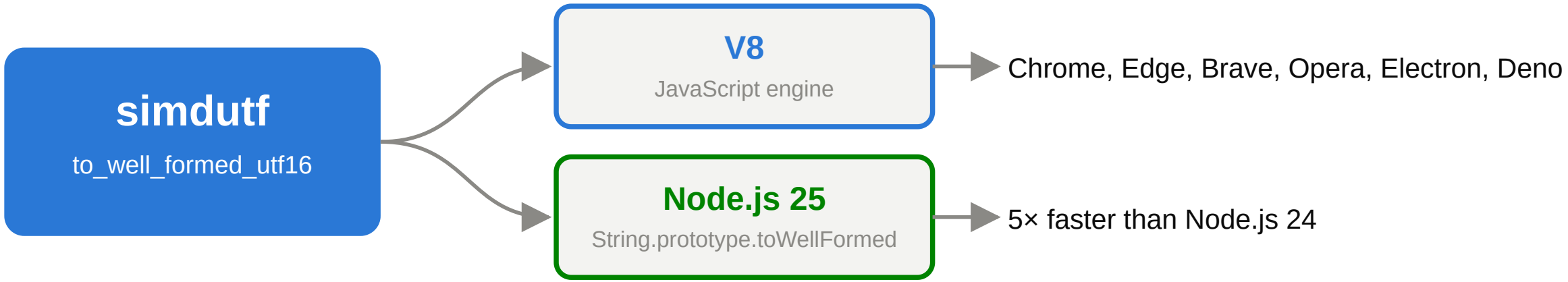
Repair, 64 bytes at a time



2 loads, 2 ANDs, 2 compares, 1 XOR, 1 test per block: no loop-carried state, no per-unit branch

idempotent, so the tail is just one more block aligned to the end of the input

One function, every V8 embedder

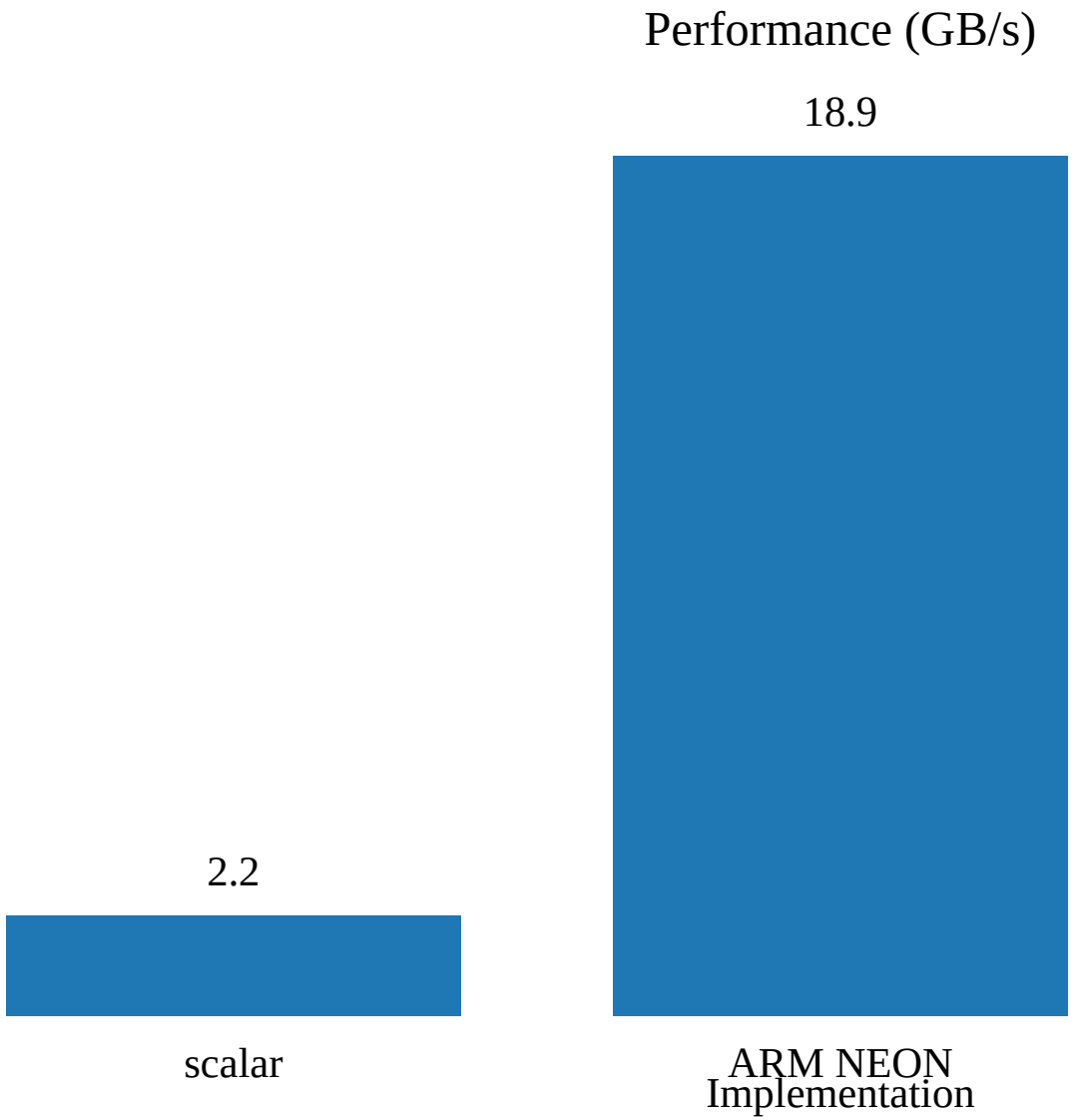


one C++ function, every V8 embedder, every well-formedness check on a JavaScript string

UTF-16 correction, Apple M4

	scalar	ARM NEON
GB/s	2.2	18.9
ins/byte	12.0	0.9

13× fewer instructions per byte.



In the browser (Apple M4)

- Chromium: 16 GB/s (**uses our new function**)
- Firefox: 3.4 GB/s
- Safari: 1.2 GB/s

Test it yourself: https://lemire.github.io/browserwellformed/

Data parallelism applies more often than you think

- JSON parsing — every JavaScript engine
- Unicode validation — every browser
- base64 — the JavaScript standard library
- Number parsing — every compiler toolchain
- Bitmap indexes, hashing, compression, `memchr`, CSV, regex prefiltering

"Inherently serial" is usually a statement about the algorithm you happen to know.

Interested? Check these projects

- **simdjson** — the fastest JSON parser in the world https://simdjson.org
  - Node.js, Bun, Deno, Electron
  - ClickHouse, WatermelonDB, Apache Doris, Meta Velox, Milvus, QuestDB, StarRocks
- **simdutf** — Unicode (UTF-8/16/32) and base64 https://github.com/simdutf/simdutf
  - Node.js, Bun, WebKit (Safari), Chromium (Chrome, Edge)
- **fast_float** — number parsing https://github.com/fastfloat/fast_float
- **Roaring Bitmaps** — https://roaringbitmap.org

Credit

- simdjson reflection work with Francisco Geiman Thiesen (Microsoft)
- simdutf UTF-16 correction is joint work with Robert Clausecker
- simdjson and simdutf are community efforts (Geoff Langdale, John Keiser, Paul Dreik, Yagiz Nizipli and others)

Extra slides

Deployed: ada → Node.js



Measured in Node.js, Xeon Gold 6548N, 100 000 URLs per corpus, best of 5:

	`new URL(s)`	`URL.canParse(s)`
IPv4 hosts	**+8%**	**+13%**
IPv6, random	**+8%**	**+20%**
IPv6, traffic-like	**+2%**	**+9%**

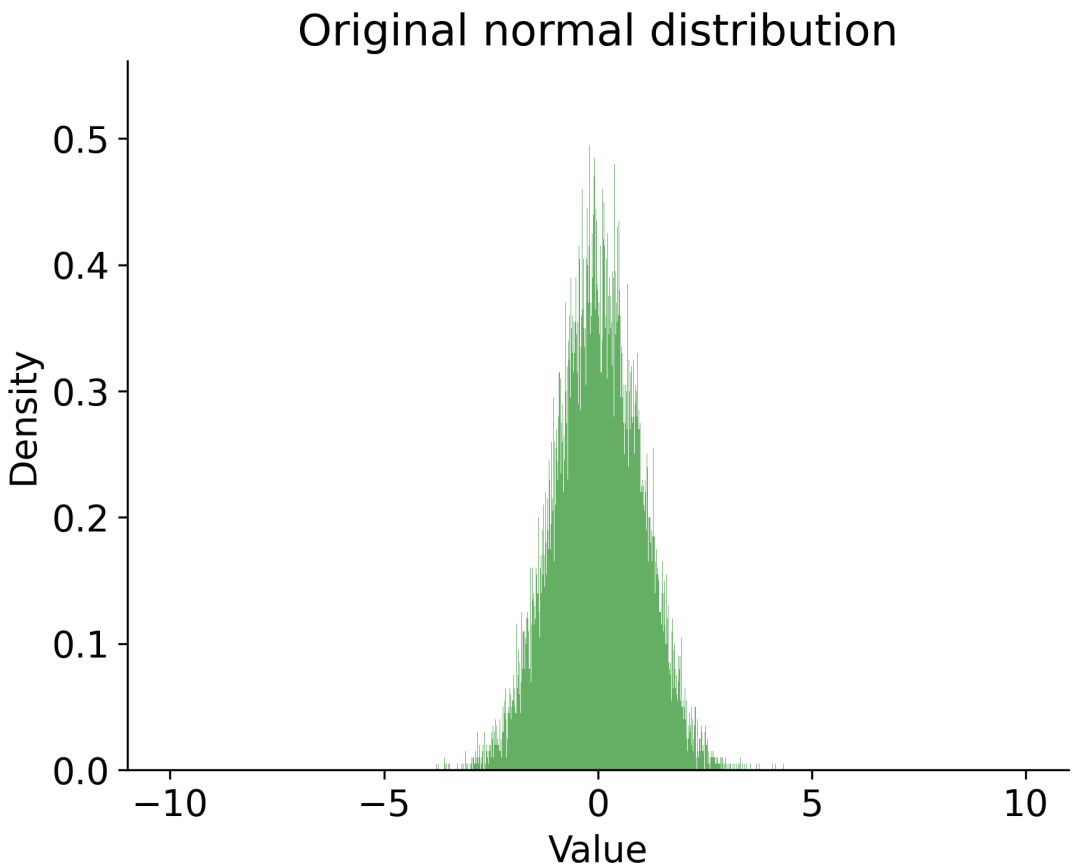
2 300–3 200 instructions per new URL; the address parser was a few hundred of them

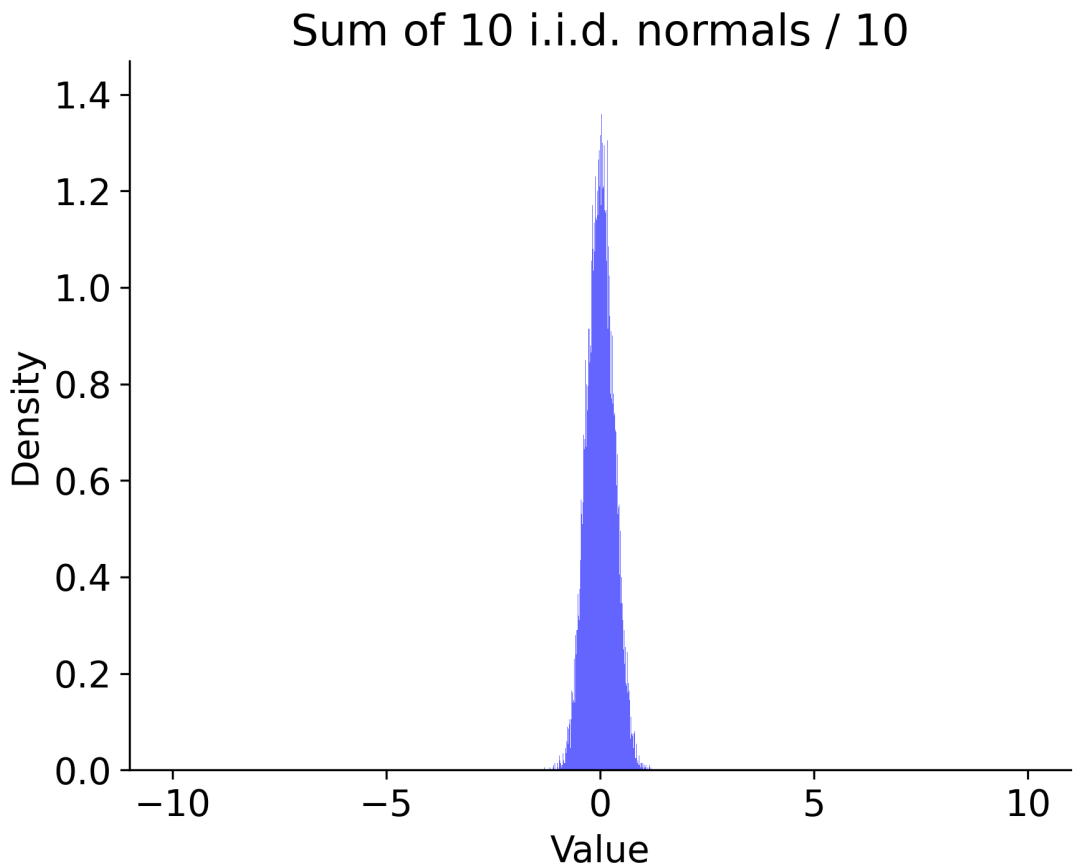
Part 7

Measure properly, or do not bother

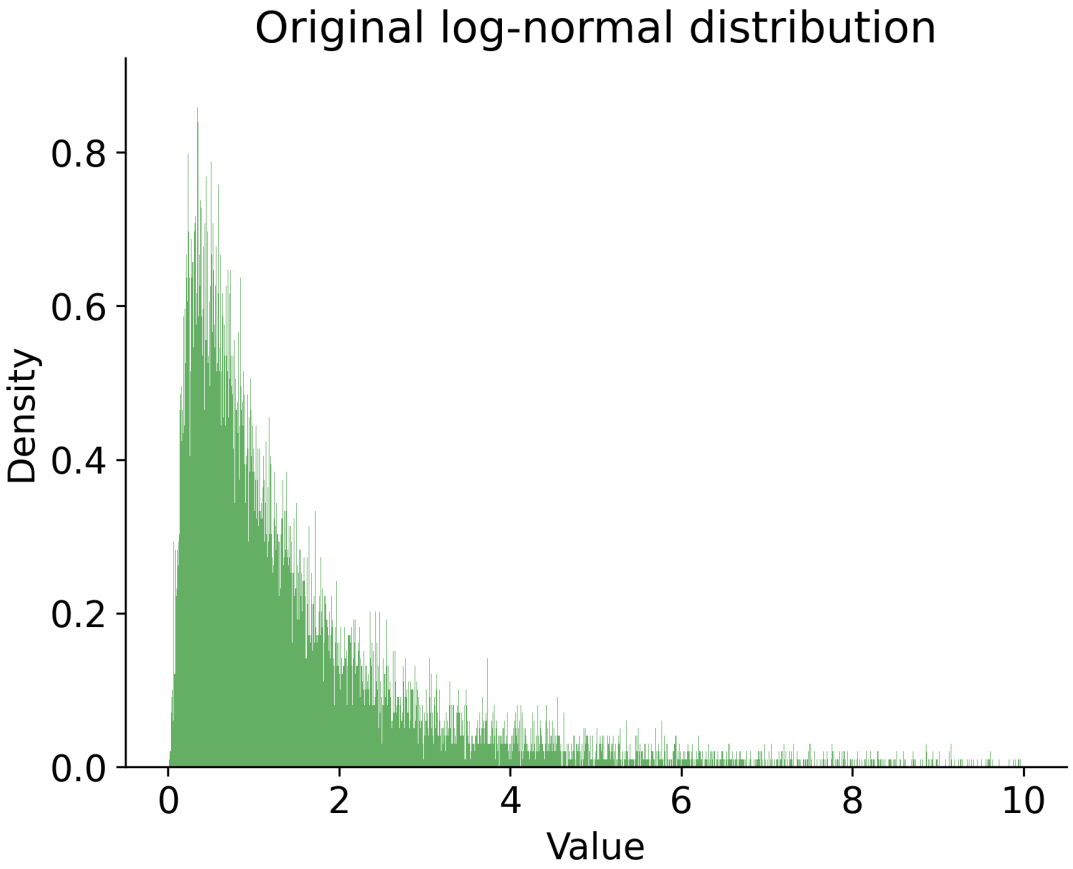
Measurements

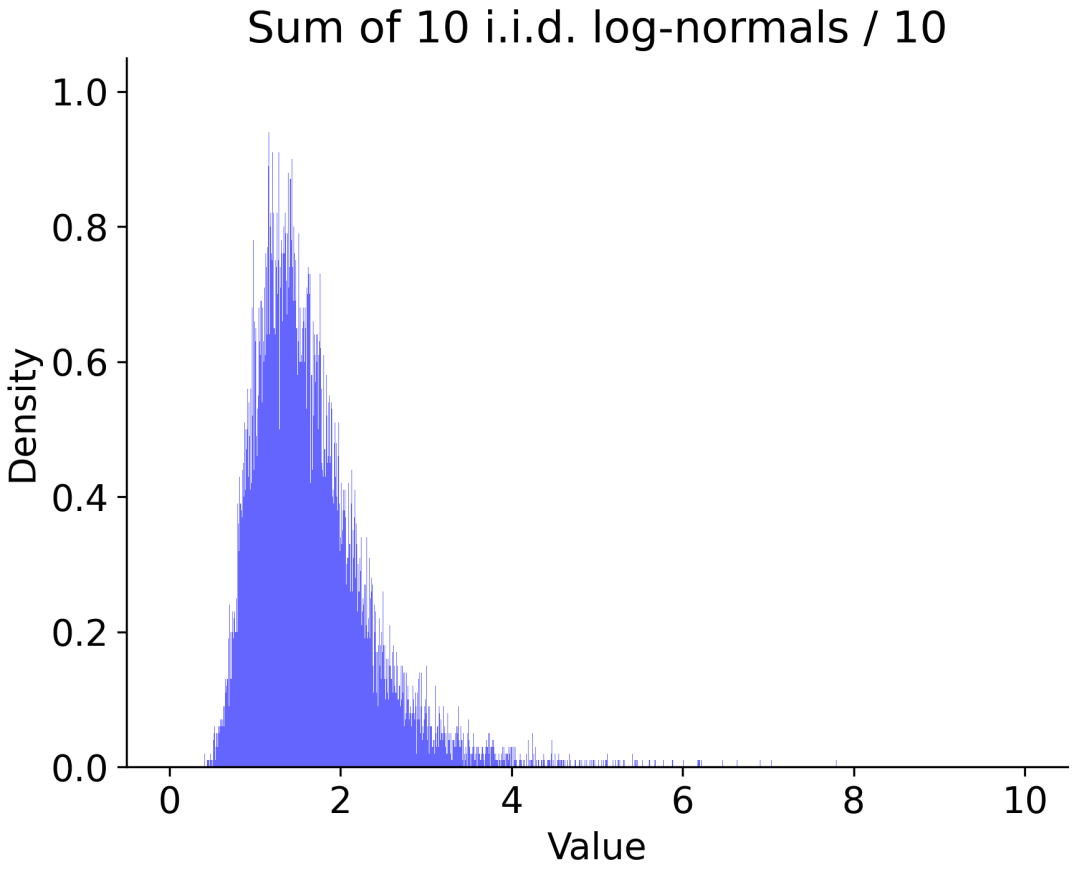
- We often assume that measurements (timings) are normally distributed.
- If they were, the 'error' would fall off as $1/\sqrt{N}$.
- It is often an incorrect assumption.


Original normal distribution
Density
0.0
0.1
0.2
0.3
0.4
0.5
−10
−5
0
5
10
Value


Sum of 10 i.i.d. normals / 10
Density
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
−10
−5
0
5
10
Value

What if we dealt with log-normal distributions?





Real-world measurements

- You cannot assume normality
- Measurements are **not independent**
- Reality: the absolute **minimum** is often the *reliable* metric
- Margin: the difference between the mean and the minimum

Use performance counters

Timings tell you *that* something is slow. Counters tell you *why*.

- **instructions retired** — did I actually remove work?
- **cycles** — the ground truth
- **branch misses** — is the predictor carrying me?
- **cache misses** — am I memory bound after all?

`perf stat`, `Instruments`, or a library such as `performancecounters`.

Hot-spot engineering fails

- A profiler shows you where the cycles are *now*.
- It does not show you the 30% of instructions spread evenly over every function.
- Reducing the total instruction count beats optimizing the top-of-profile function.

Part 6

Can an LLM write your SIMD code?

Partly

Frontier models in 2026 are genuinely good at:

- Recalling intrinsic names and semantics (better than I am)
- Translating a working NEON kernel to AVX2, or to RVV
- Writing the scalar reference implementation and the test harness
- Explaining an unfamiliar instruction

What actually works: close the loop

Help your agent.

1. **A benchmark** it can run, over realistic inputs
2. **A differential fuzzer** against a scalar reference
3. `llvm-mca` (or `perf`) so it can see cycles, not vibes

Then let it iterate.

Case study: perfect hashing

Looking up!

- Map a string to a value: HTTP method, header name, URL scheme, keyword, MIME type
- The keys are **fixed when you write the code**
- Yet we hash, mask, probe, chase a pointer, compare: `std::unordered_map`, 12.8 ns
- With the keys known at compile time, we can build a **perfect hash**: no collisions, one candidate per slot

Library: github.com/ConstexprCore/perfect_hash (with Francisco Geiman Thiesen at Microsoft)

One hash, one comparison

```
slot = (len + asso[key[0]]) & 7
```

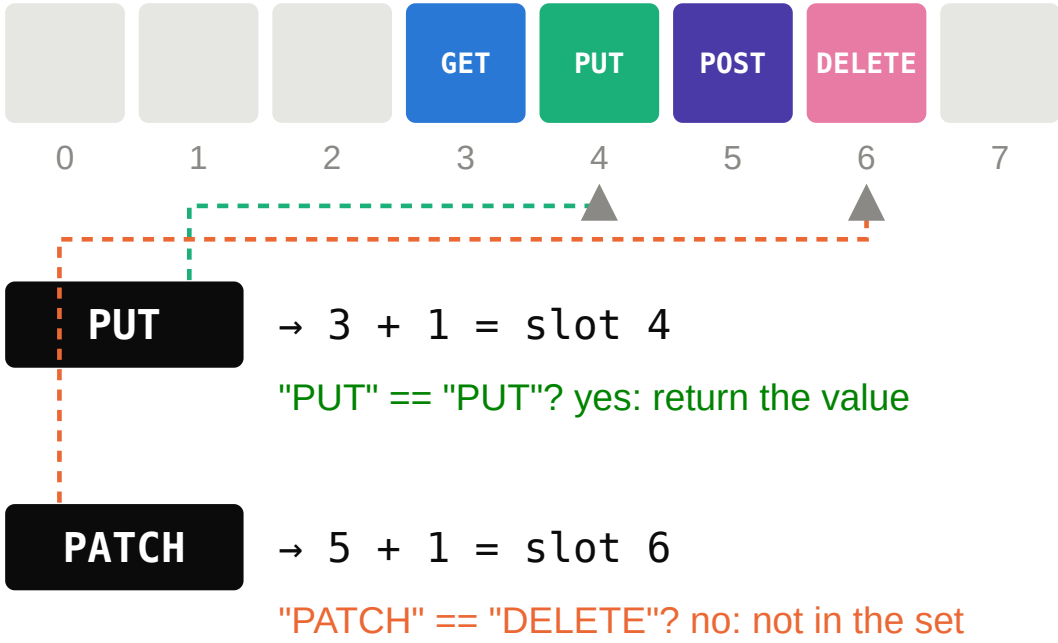
keys, known at compile time

key	len	key[0]	asso	slot
GET	3	G	0	**3**
PUT	3	P	1	**4**
POST	4	P	1	**5**
DELETE	6	D	0	**6**

`asso: G → 0, P → 1, D → 0`

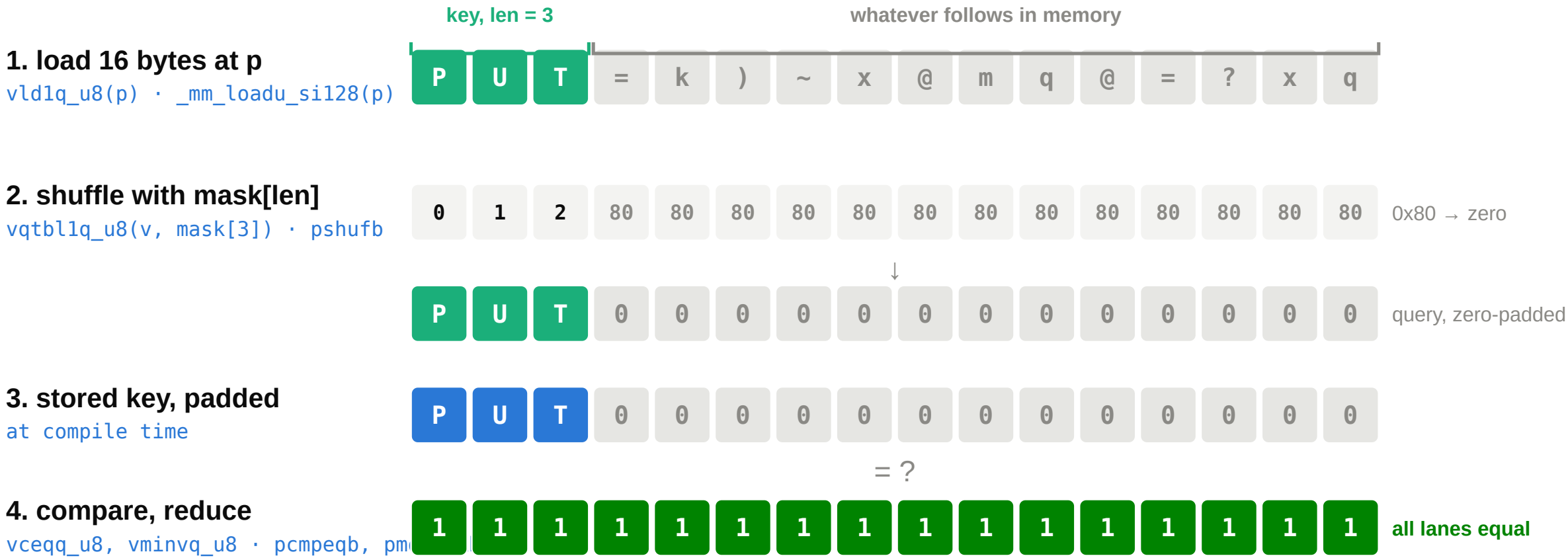
found by a bounded greedy search
at compile time (constexpr)

8 slots, one candidate each



one hash, one comparison, no probing, no chain

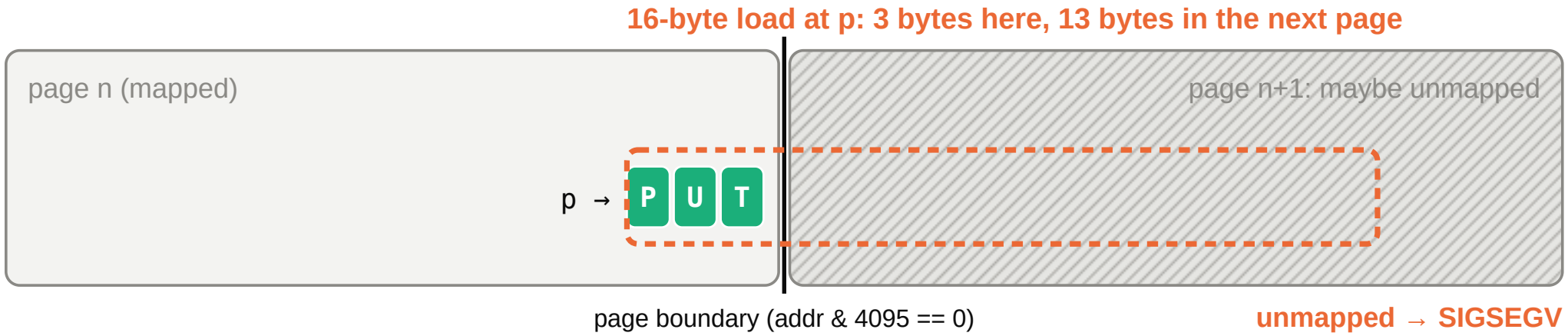
Compare the whole key at once



byte loop: ~40 instructions, up to 8 branches → 5 instructions, 0 branches, every key, every length

The memory page trick

memory, in 4 KB pages



a load that stays inside one page cannot fault

the OS maps memory page by page; the protection unit is 4 KB, never a single byte

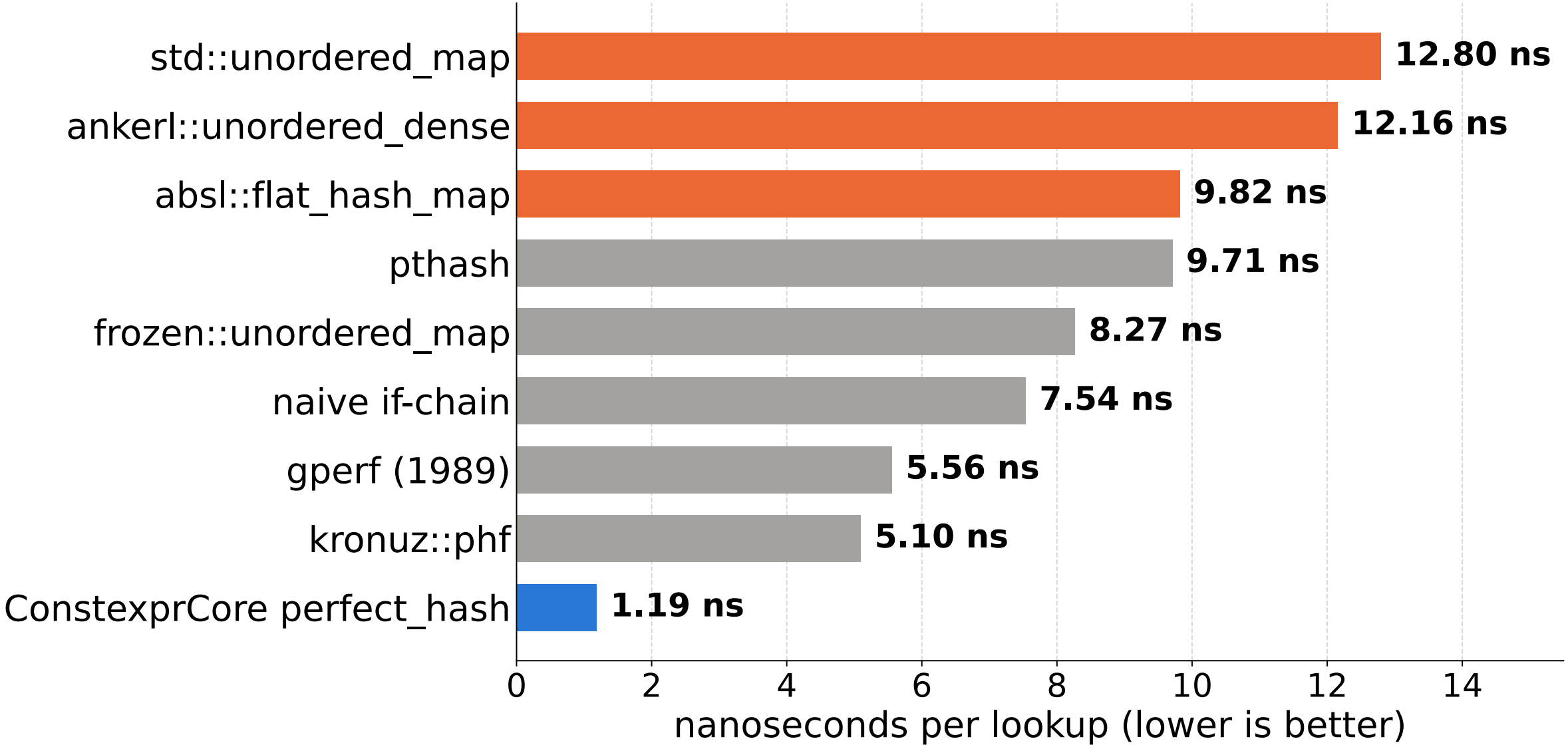
```
if ((addr & 4095) <= 4096 - 16) [[likely]]
    return load16(p); // wide, branch-free
copy len bytes to a 16-byte buffer; // rare
return load16(buffer);
```

one branch, taken 99.6% of the time

(15 of every 4096 addresses)

the predictor learns it instantly;
8 unpredictable branches → 1 predictable one

URL schemes, 6 keys, Apple M3 Max



1.19 ns, 4.9 cycles, 8.7 instructions per cycle, **zero** branch mispredictions.

Case study: base64

Base64

- Encodes binary data as text using 64 characters (A-Z, a-z, 0-9, +, /)
- 3 bytes input → 4 characters output (33% overhead)
- Data URLs, email, JWTs, web APIs, embedded images
- `"Hello, World!"` → `SGVsbG8sIFdvcmxkIQ==`

Bit manipulation on a fixed schedule: **the ideal SIMD problem.**

New JavaScript functions

```
const b64 = Uint8Array.prototype.toBase64(bytes);
const recovered = Uint8Array.fromBase64(b64);
```

function (Safari, Apple M4)	speed
`Uint8Array.fromBase64()`	11 GiB/s
`Uint8Array.toBase64()`	20 GiB/s

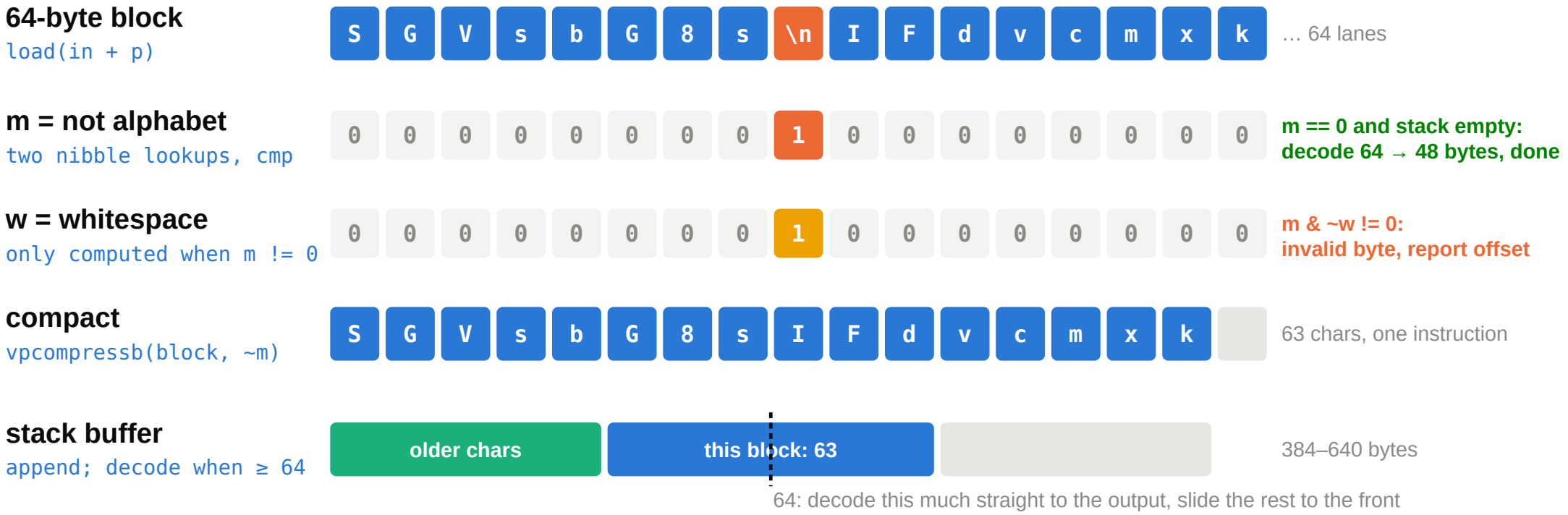
Test in your browser: https://simdutf.github.io/browserbase64/

The web's Base64 is not RFC 4648

- ES2026 `Uint8Array.fromBase64` follows WHATWG *forgiving base64*: **ignore** ASCII whitespace, **reject** anything else
- MIME e-mail breaks lines every 76 characters; DNS zone files put spaces every 50; JSON and data URLs wrap freely
- Every earlier SIMD decoder assumed clean input: one `\n` and it falls back to a byte loop

Keep the SIMD throughput on input that has whitespace in it.

Decode in blocks, compact the rare ones



clean input

no whitespace: every block takes the fast path; the stack buffer is never touched

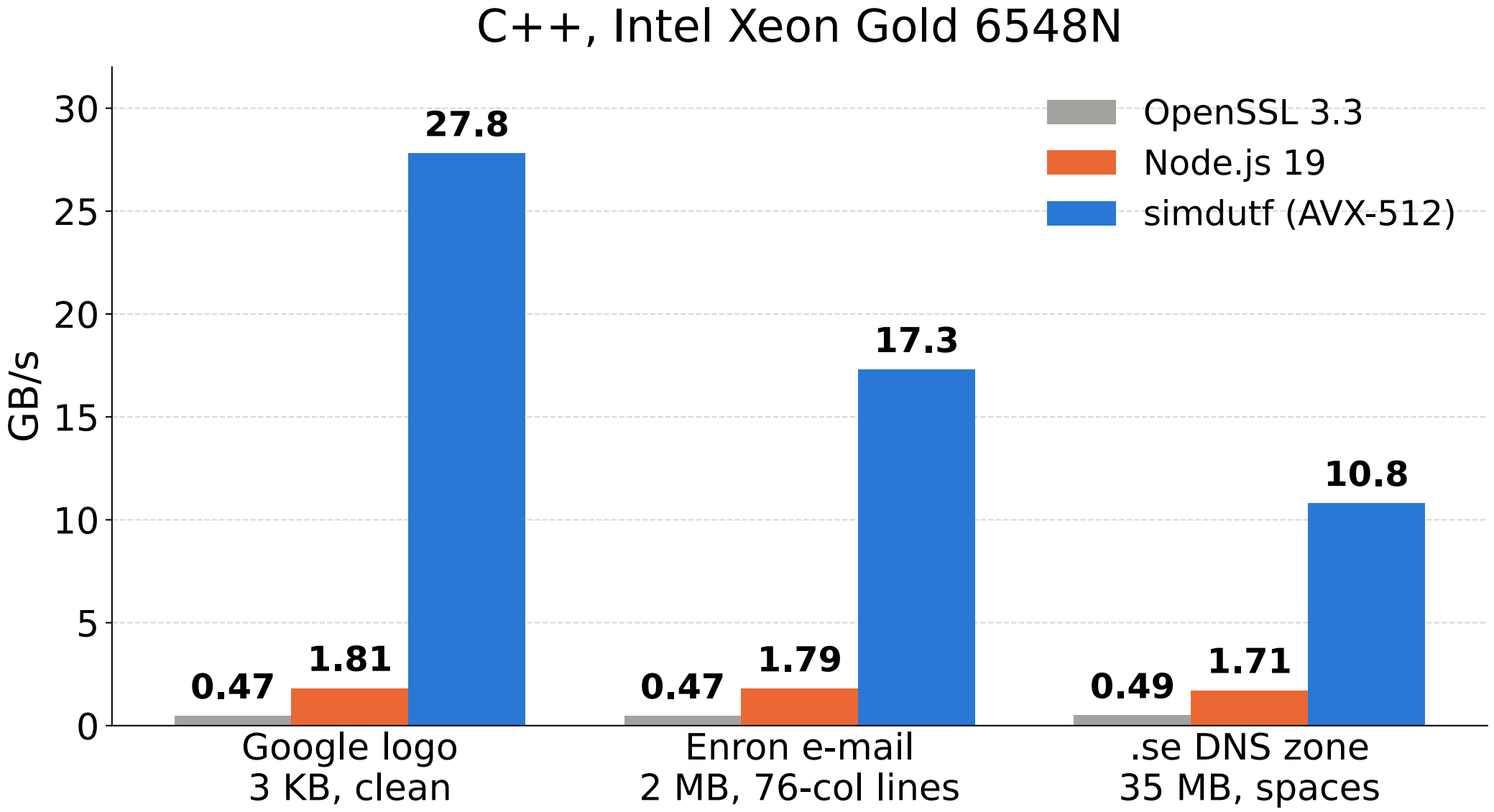
MIME / DNS / JSON

a line break every 76 chars: one compaction per block, still SIMD throughput

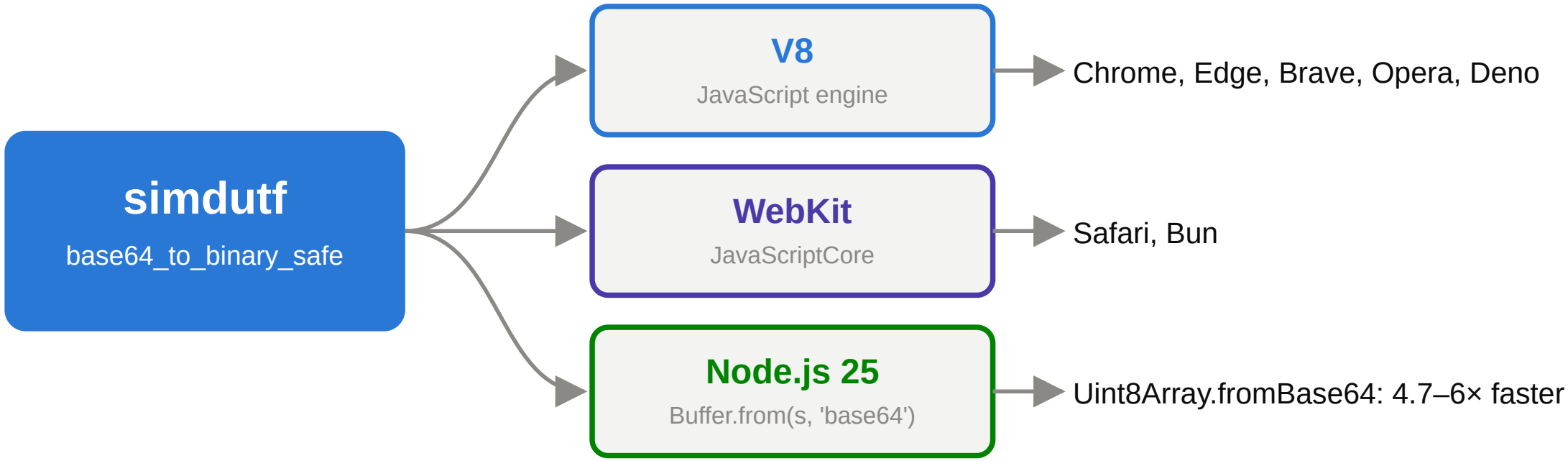
garbage

first byte that is neither alphabet nor whitespace → error with its offset

Base64 decoding with whitespace



In the browsers, in the runtimes



Uint8Array.fromBase64 / toBase64 landed in ES2026; the decoder underneath is this one

AVX-512 base64 encoding

- Encoding a 64-byte block requires only **two** non-memory instructions:
  `vpermb` (twice) and `vpmultishiftqb`.
- The right instruction turns an algorithm into a lookup.

